



An enhanced genetic algorithm for optimized task allocation and planning in heterogeneous multi-robot systems

Ahmed Nait Chabane¹ · Ouahib Guenounou²

Received: 19 April 2025 / Accepted: 11 August 2025 / Published online: 2 September 2025
© The Author(s) 2025

Abstract

Efficient task allocation and path planning in heterogeneous multi-robot systems (MRS) remains a significant challenge in industrial inspection contexts, particularly when robots exhibit diverse sensing capabilities and must operate across spatially distributed sites. To address the limitations of exact methods and conventional heuristics, we propose a novel two-phase enhanced genetic algorithm (EGA) tailored for capability-constrained task assignment and route optimization. The first phase employs a domain-specific chromosome encoding to assign tasks while enforcing robot-measurement compatibility. The second phase locally refines each robot's path to minimize travel distance and improve load balancing. We benchmark the EGA against an exact mixed integer linear programming (MILP) model, a standard genetic algorithm (single-phase), a particle swarm optimization (PSO) approach, and an adapted version of the bi-level surrogate-assisted evolutionary algorithm (BL-SAEA) across scenarios involving up to 50 inspection sites and 4 heterogeneous robots. Experimental results show that our EGA consistently produces near-optimal solutions, achieving average optimality gaps below 1.5%, while reducing computation times by up to 90% compared to MILP. Furthermore, the second phase significantly enhances convergence stability and solution robustness, especially in large-scale instances. These results demonstrate the scalability and practical suitability of the proposed method for real-time, resource-constrained industrial inspection missions.

Keywords Multi-robot task allocation (MRTA) · Genetic algorithm (GA) · Mixed integer linear programming (MILP) · Industrial inspection

Introduction

Multi-robot systems (MRS) have become increasingly essential in industrial operations such as inspection, maintenance, and monitoring, particularly within the context of Industry 4.0 and the emerging paradigm of Industry 5.0 [1, 2]. While Industry 4.0 emphasizes automation and data-driven decision-making, Industry 5.0 builds upon these foundations by promoting closer collaboration between humans and robots to achieve both productivity and human-centric

objectives [3]. Despite these advancements, coordinating multiple robots in complex industrial environments remains a significant challenge, especially when robots possess heterogeneous capabilities and tasks require diverse sensing or action types [4].

A major difficulty lies in the multi-robot task allocation (MRTA) and planning problem, which involves assigning tasks to robots and determining their optimal execution sequence. This challenge is further exacerbated in heterogeneous MRS, where robots may differ in sensor suites, manipulation capabilities, and energy constraints. Consequently, suboptimal task allocation can result in unnecessary downtime, increased energy consumption, and even safety risks [5, 6]. Industries operating over geographically distributed sites—such as oil and gas pipeline inspections, manufacturing plant maintenance, and logistics warehouses—especially require robust and scalable optimization techniques to address these issues.

Although exact methods such as mixed integer linear programming (MILP) are theoretically attractive for generat-

Ahmed Nait Chabane and Ouahib Guenounou have contributed equally to this work.

✉ Ahmed Nait Chabane
anaitchabane@cesi.fr

Ouahib Guenounou
oguenounou@cesi.fr

¹ CESI LINEACT, 24 Le Paquebot, Saint Nazaire, France

² CESI LINEACT, 44 Frédéric Auguste Bartholdi, Le Mans, France

ing optimal solutions, they quickly become computationally intractable for large-scale or real-time applications [7]. In contrast, metaheuristic approaches—particularly genetic algorithms (GA)—offer a more scalable alternative for exploring vast solution spaces. Inspired by biological evolution, GAs can flexibly accommodate heterogeneous robot constraints and inter-task dependencies [8, 9]. However, standard GAs often suffer from slow convergence and limited solution quality, especially in dynamic industrial settings where task priorities, energy availability, or environmental conditions may vary [10–12].

To address these limitations, we propose a two-phase enhanced genetic algorithm (EGA) specifically designed for heterogeneous multi-robot inspection tasks. Our approach integrates domain-specific genetic operators and a custom chromosome encoding scheme that effectively captures industrial constraints. The first phase assigns tasks across the robot fleet, ensuring feasibility while minimizing total travel distance for a fixed number of robots. The second phase refines each robot's individual route, further reducing travel distance and improving workload balance under changing operational conditions. This two-phase strategy simultaneously addresses system-level scalability (Phase 1) and local route optimization (Phase 2), thereby enhancing the robustness of solutions in large-scale or evolving environments.

We benchmark the performance of the proposed EGA against an exact optimization model and three representative metaheuristic approaches across multiple real-world-inspired scenarios varying in scale, robot capabilities, and spatial task distribution. Experimental results show that the EGA consistently yields near-optimal solutions at a fraction of the computational cost of the exact model, particularly in large-scale instances. Moreover, when compared to the other metaheuristic approaches, the two-phase architecture achieves superior solution quality. The key contributions of this paper are as follows:

1. A comprehensive formulation of the MRTA and planning problem for heterogeneous MRS, capturing the complexity introduced by robot capability diversity and task interdependencies in industrial environments.
2. A two-phase enhanced genetic algorithm featuring customized genetic operators and a tailored encoding scheme that ensures feasibility, reduces total travel distance, and scales efficiently.
3. An extensive performance evaluation was conducted, comparing EGA with an exact MILP formulation and other representative metaheuristic approaches in various scenarios, highlighting gains in computational efficiency and solution quality.

The remainder of this paper is organized as follows. Section “[Related work](#)” reviews the relevant literature on MRTA

and metaheuristic optimization. Section “[Problem formulation](#)” formalizes the MILP model employed as the exact optimization method for benchmarking. Section “[Methodology proposed using an enhanced genetic algorithm](#)” details the proposed two-phase EGA methodology. Section “[Experimental plan and use case](#)” presents the experimental setup and a real-world use case in multi-robot inspection. Section “[Results and discussion](#)” discusses the results and provides a detailed comparative analysis. Section “[Scalability and computational-complexity analysis](#)” evaluates the scalability and computational complexity of the proposed approach. Section “[EGA performance in a robot breakdown scenario](#)” analyzes EGA performance in a robot breakdown scenario. Finally, Sect. “[Conclusion and future work](#)” concludes the paper and outlines directions for future research.

Related work

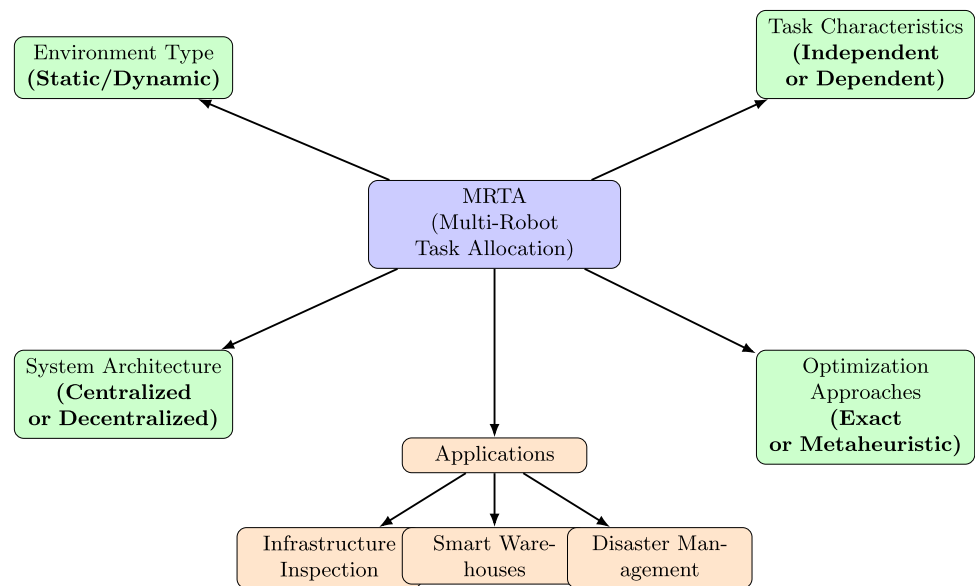
The task allocation problem in multi-robot systems (MRS) has been extensively studied in the robotics and operations research communities, particularly in the context of multi-agent systems (MAS) [13, 14]. This section reviews the most relevant work on task allocation in MRS, focusing on heterogeneous robots and evolutionary algorithms.

Task allocation in multi-robot systems

The task allocation problem in MRS involves distributing tasks across a fleet of robots to optimize objectives such as energy consumption, task completion time, or distance traveled. Gerkey and Mataric [15] provided a comprehensive taxonomy of task allocation problems in MRS, categorizing them according to task interdependencies and assignment strategies. Centralized approaches [16, 17], such as auction-based algorithms [18], are effective in generating optimal solutions, but are often limited by scalability issues as the number of robots and tasks grows.

Decentralized approaches, including market-based systems [19–21], distribute the decision-making process among robots, increasing scalability and fault tolerance. However, these methods typically result in suboptimal solutions due to the lack of global information and coordination. Several decentralized methods have been surveyed in [21, 22], acknowledging their scalability but highlighting difficulties in handling complex task dependencies, especially in heterogeneous systems. Despite progress, most of these methods do not adequately address the specific challenges of heterogeneous robots with varying capabilities, which is the focus of our study. Figure 1 presents a comprehensive taxonomy of multi-robot task allocation (MRTA) systems, highlighting the critical dimensions and their interrelations. At the core lies the MRTA concept, which serves as the central frame-

Fig. 1 Taxonomy of multi-robot task allocation (MRTA). The diagram illustrates the key components, including environment types, task characteristics, system architecture, optimization approaches, and practical applications



work for understanding the distribution and coordination of tasks among multiple robots. The taxonomy is divided into five major components:

- **Environment type:** This dimension differentiates between static and dynamic environments. Static environments assume fixed conditions throughout task execution, whereas dynamic environments account for changing conditions, such as the appearance of obstacles or evolving task priorities.
- **Task characteristics:** This branch categorizes tasks as independent, where tasks can be executed in isolation, or dependent, where task completion requires precedence or collaborative execution.
- **System architecture:** This component examines whether task allocation is managed through a centralized control system, offering global optimization but limited scalability, or a decentralized architecture, enhancing fault tolerance and scalability at the cost of potential suboptimality.
- **Optimization approaches:** This category contrasts exact methods, such as mixed integer linear programming (MILP), which provide optimality guarantees at the cost of high computational effort, with metaheuristic approaches, such as genetic algorithms and particle swarm optimization, which offer scalable and near-optimal solutions for complex problems.
- **Applications:** Practical applications of MRTA are shown, including infrastructure inspection, smart warehouses, and disaster management, demonstrating the applicability and relevance of MRTA systems in real-world scenarios.

Heterogeneous robot systems

When robots have different sensor suites, locomotion mechanisms, or energy constraints, the task allocation process becomes substantially more complex. Notomista et al. [23] examined a distributed task allocation strategy tailored for heterogeneous MRS, highlighting how robot diversity can complicate standard allocation mechanisms.

Subsequent work introduced approaches such as capability-based allocation [24] and skill matching algorithms [25], where robots are matched to tasks based on proficiency or sensor compatibility. However, most of these methods assume relatively static environments, sidestepping key challenges relevant to large-scale or dynamic industrial scenarios (e.g., sudden equipment failures, evolving task requirements, or constrained energy resources). The growing need to dispatch robots across spatially dispersed sites for specialized tasks further underscores the importance of robust and scalable allocation algorithms.

In the context of safe and intelligent task execution by UAVs, Jiang et al. [26] proposed a distributed ranging system that leverages intelligent sensor integration to ensure safe UAV navigation and distribution. Their architecture combines adaptive perception with coordination rules, which is particularly relevant to inspection scenarios requiring dynamic reactivity and spatial awareness. This work complements our focus by addressing real-time safety and multi-agent awareness through sensor fusion, and highlights the importance of robust low-level sensory frameworks to support high-level task allocation algorithms.

Recent advances in intelligent control architectures have further emphasized the synergy between optimization and adaptive learning. For instance, Puentes et al. [27] proposed

a neural network-based controller framework to enhance real-time adaptability in robotic control systems, a direction that complements our high-level decision optimization. Similarly, Li and Shen [28] illustrated how robotics and structural optimization can be integrated in civil engineering scenarios, demonstrating the relevance of scalable task planning in domains where spatial constraints and robot-enabled inspection or intervention are critical. These contributions reinforce the importance of flexible and modular frameworks for multi-robot coordination under diverse operational constraints.

Evolutionary algorithms for task allocation

Evolutionary algorithms (EAs) draw inspiration from natural processes to address complex optimization challenges in multi-robot systems (MRS). Examples of widely studied EAs include ant colony optimization (ACO), particle swarm optimization (PSO), and genetic algorithms (GA) [29]. These techniques have been applied extensively to the multi-robot task allocation (MRTA) problem, particularly in industrial and cooperative settings where scalability and reliability are paramount.

Ant colony optimization (ACO)

Liu et al. [30] developed an efficient ant colony system (ACS) tailored for the MRTA problem, particularly focusing on scenarios involving large-scale cooperative tasks with precedence constraints. Their approach introduced a novel task-alliance encoding scheme and a combination of pheromone and heuristic information, allowing the ACS to effectively explore the solution space and construct high-quality task allocation schedules. Experimental results demonstrated that the proposed ACS outperformed traditional heuristic methods in scalability and solution quality, particularly for complex problem instances. However, the method's reliance on predefined parameters and its computational demands in high-dimensional settings may limit real-time adaptability in highly dynamic industrial environments.

Particle swarm optimization (PSO)

PSO has been widely applied to MRS task scheduling due to its simplicity and efficiency in exploring large solution spaces [31]. Kong et al. proposed a hybrid PSO algorithm with an enhanced fitness evaluation function to optimize task assignments while addressing constraints such as task priorities and communication delays. Their method demonstrated faster convergence and better solution quality compared to standard PSO. However, the performance of PSO can degrade in heterogeneous systems with high task dependencies, where robot-specific capabilities and complex intertask

relationships introduce additional challenges. These limitations emphasize the need for further refinement of PSO-based methods, such as integrating domain-specific heuristics or hybridizing PSO with other metaheuristic algorithms, to effectively manage heterogeneity and dependencies between tasks in MRS.

Genetic algorithms (GA)

Genetic algorithms (GAs) are acknowledged as a versatile and robust metaheuristic for tackling intricate optimization problems, such as task allocation and planning in multi-robot systems (MRS). Using evolutionary principles such as selection, crossover, and mutation, GAs effectively explore extensive solution spaces, making them ideal for scenarios with diverse robot capabilities and dynamic constraints [32]. Their support for multi-objective optimization aligns well with goals such as minimizing travel distance, balancing workload, and meeting energy constraints [33]. Recent research highlights the incorporation of problem-specific constraints in GAs. Zhang et al. [34] refined a GA with time window restrictions in the fitness function, enhancing task deadline management in MRS. Although their method performed well in medium-sized scenarios, it showed reduced computational efficiency as the number of robots and tasks grew, pointing out scalability issues. Managing heterogeneity in MRS adds complexity. According to [35], a capability-based chromosome encoding scheme was proposed to accommodate variations in sensing, locomotion, and operational constraints among robots, showing improvements over conventional methods in simulations. However, it did not address real-world complexities such as stochastic task influx, real-time adaptability, and strict energy limits common in industrial settings. Addressing such challenges is critical for effective task allocation in heterogeneous MRS.

Recent progress highlights the importance of crossover operators in improving GA performance for task allocation in MRS. Tailored crossover mechanisms influence solution quality and convergence by optimizing search space exploration. Mammadzada et al. [35] introduced capability-aware chromosome encoding, emphasizing the need for GAs adjusted to robotic heterogeneity, which improved performance in simulated settings but lacked real-world applicability due to the lack of dynamic constraint support. Furthermore, Shem-Tov et al. [36] developed the deep neural crossover (DNC), utilizing deep reinforcement learning and recurrent neural networks to refine gene selection in crossover operations. Unlike traditional methods, DNC dynamically learns gene correlations, enhancing offspring quality by prioritizing robust genetic combinations, outperforming standard methods like one-point and uniform crossovers in fitness and convergence speed. These insights

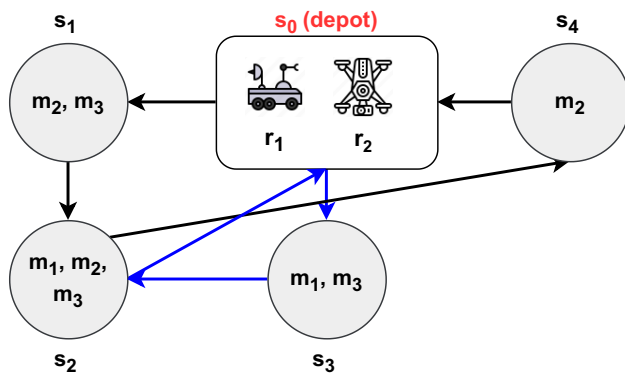


Fig. 2 Example trajectories of robots r_1 (black) and r_2 (blue). Sites s_i are marked for inspection, with the measurements m_i performed by the corresponding robots

emphasize the necessity of developing crossover operators tailored to the specific requirements of heterogeneous MRS task allocation.

Problem formulation

Multi-robot system description

We consider a team of R heterogeneous robots $\{r_1, r_2, \dots, r_R\}$, each equipped with specific sensor capabilities, tasked with performing industrial site inspections. The system operates over a set of N distinct sites $\{s_1, s_2, \dots, s_N\}$ in addition to a depot s_0 where all robots start and finish their missions.

At each site s_i ($i = 1, \dots, N$), a selection of measurements from $M = \{m_1, m_2, \dots, m_M\}$ is needed. Owing to the heterogeneous nature of the robots, not all robots can perform every measurement; each robot r_k possesses a specific set of capabilities $C_k \subseteq M$. Consequently, the task allocation and planning process must simultaneously consider the robots' capability constraints and the objective of minimizing the total travel distance. Assuming that energy consumption is primarily related to movement, this objective inherently contributes to reducing the overall energy consumption.

Figure 2 presents a scenario with two potential robot paths for r_1 and r_2 . As we can see, the robot r_1 moves from the depot site s_0 and visits successively s_1, s_2 , and s_4 to perform, respectively, measurements $\{m_2, m_3\}$ in s_1 and $\{m_2, m_3\}$ again in s_2 and only m_2 in s_4 before returning to the depot s_0 . The robot r_2 first visits s_3 to perform $\{m_1, m_3\}$, followed by the site s_2 to perform m_1 , and returns to the depot. For this example, we have considered that robot r_1 is capable of performing measurements m_2 and m_3 , while robot r_2 performs measurements m_1 and m_3 .

Mathematical model

In this section we present the formulation model of the MRS problem. We give decision variables, the objective function and constraints equations.

Decision variables

- $x_{ijr} \in \{0, 1\}$: Equals 1 if robot r travels directly from site s_i to site s_j ; 0 otherwise.
- $y_{ir} \in \{0, 1\}$: Equals 1 if robot r visits site s_i ; 0 otherwise.
- $z_r \in \{0, 1\}$: Equals 1 if robot r is deployed (i.e., it visits at least one site); 0 otherwise.
- $u_{ir} \in \mathbb{R}^+$: A continuous variable used for subtour elimination via the Miller–Tucker–Zemlin (MTZ) formulation for each site s_i ($i = 1, \dots, N$) and each robot $r = 1, \dots, R$.

Objective function

The objective is to minimize the total travel distance of all robots, formulated as:

$$\min \sum_{r=1}^R \sum_{i=0}^N \sum_{j=0}^N d_{ij} x_{ijr}, \quad (1)$$

where d_{ij} is the Euclidean distance between sites s_i and s_j , and the binary decision variable $x_{ijr} = 1$ if robot r travels directly from site s_i to site s_j , and 0 otherwise.

Constraints

(1) *Linking constraint* A robot may only traverse an arc if it is deployed:

$$x_{ijr} \leq z_r, \quad \forall r \in \{1, \dots, R\}, \quad \forall i, j \in \{0, \dots, N\}. \quad (2)$$

(2) *Depot departure and return* For each robot r :

$$\begin{cases} \sum_{j=1}^N x_{0jr} = y_{0r} \\ \sum_{i=1}^N x_{i0r} = y_{0r} \end{cases} \quad (3)$$

These ensure each robot leaves the depot exactly once (if it is used) and returns exactly once.

(3) *Flow conservation* For each non-depot site s_i (with $1 \leq i \leq N$) and each robot r :

$$\begin{cases} \sum_{\substack{j=0 \\ j \neq i}}^N x_{ijr} = y_{ir} \\ \sum_{\substack{j=0 \\ j \neq i}}^N x_{jir} = y_{ir} \end{cases} \quad (4)$$

These constraints ensure that if a robot visits site s_i , it must both enter and leave that site.

(4) *Site visit requirement* Each site s_i (with $1 \leq i \leq N$) must be visited by at least one robot:

$$\sum_{r=1}^R y_{ir} \geq 1, \quad i = 1, \dots, N. \quad (5)$$

(5) *Measurement coverage* For each site s_i (with $i \geq 1$) and each measurement $m \in M_i$, at least one capable robot must be assigned:

$$\sum_{r \in R_m} \sum_{j=0}^N x_{jir} \geq 1, \quad i = 1, \dots, N; \quad m \in M_i, \quad (6)$$

where $R_m = \{r \in \{1, \dots, R\} \mid m \in C_r\}$ is the set of robots able to perform measurement m .

(6) *Subtour elimination (MTZ)* To eliminate subtours, for every robot r and all distinct i, j with $1 \leq i < j \leq N$:

$$u_{ir} - u_{jr} + (N+1)x_{ijr} \leq N, \quad 1 \leq u_{ir} \leq N. \quad (7)$$

(7) *No self-loops*

$$x_{iir} = 0, \quad \forall i = 0, \dots, N; \quad r = 1, \dots, R. \quad (8)$$

A robot cannot travel from a site back to the same site directly.

(8) *Linking y and z* A robot can visit site s_i only if it is deployed:

$$y_{ir} \leq z_r, \quad \forall i \in \{0, \dots, N\}, \quad r \in \{1, \dots, R\}. \quad (9)$$

Complexity analysis

The task allocation and planning problem described above is NP-hard due to its combinatorial nature [15]. The challenge lies in assigning multiple tasks to a set of robots while considering individual robot capabilities, minimizing the distance traveled, and adhering to operational constraints such as energy limitations. The number of possible combinations increases exponentially as the number of robots and sites increases, making exact solutions computationally prohibitive for large-scale problems.

Exact methods, such as mixed integer linear programming (MILP), can yield optimal solutions, but their use is

limited to small-scale instances due to high computational costs [37]. For large-scale and real-time industrial applications, metaheuristic approaches such as genetic algorithms (GA) are more suitable [32], as they can find near-optimal solutions within practical time frames. These algorithms efficiently explore large search spaces by evolving solutions over successive generations, using genetic operators such as crossover, mutation, and selection.

Methodology proposed using an enhanced genetic algorithm

This section outlines our method for addressing task allocation and planning in multi-robot systems, as depicted in Fig. 3. The approach is split into two phases. The first phase uses a genetic algorithm to initially optimize task assignments and plans, considering robot capabilities and system goals, such as site-specific measurements. It applies selection, crossover, and mutation processes. The second phase refines each robot's task plan with a targeted genetic algorithm, enhancing path optimization. As detailed later, this step is less complex because task assignments are already determined, allowing robots to optimize the path distance locally.

The main notations used throughout this paper, particularly in the pseudocode of the proposed approach (Algorithms 1–3), are summarized in Table 1 for clarity and reproducibility.

Phase 1: global task allocation and planning

The approach follows steps similar to the standard genetic algorithm, but with significant modifications in the encoding procedure and genetic operators. The process begins with the generation of an initial population, followed by an evaluation procedure, then selection and genetic operations (crossover and mutation) are applied to generate the next generation. The procedure is repeated until the termination criterion is reached, at which point the optimal solution is recorded.

Initial population

Before generating the initial population, we first define the chromosome structure, represented as a matrix with three rows and a variable number of columns. The first row specifies the site index where each measurement is performed, the second row holds the measurement index, and the third row indicates the robot responsible for that measurement. Figure 4 illustrates this chromosome structure for the scenario shown in Fig. 2. For example, the yellow column indicates that robot r_2 performs measurement m_1 at site s_3 . In the

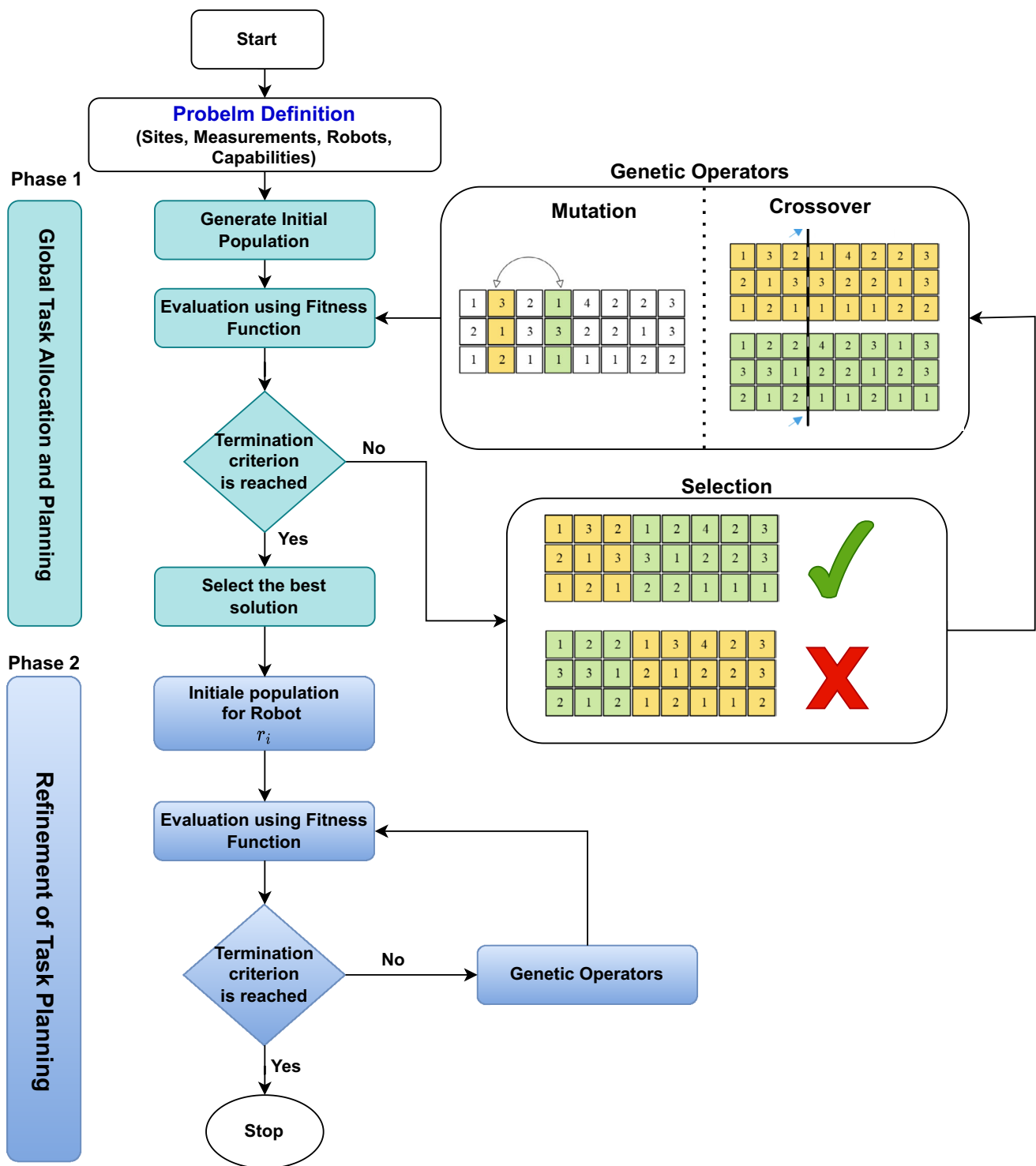


Fig. 3 Proposed enhanced genetic algorithm (EGA) based on two phases

Table 1 Notations and symbols used in the proposed algorithm

Symbol/term	Description
Parent1, Parent2	Parent chromosomes selected for crossover
Offspring1, Offspring2	New chromosomes generated after crossover
P_c	Crossover probability
P_m	Mutation probability
CrossoverPoint	Random index where crossover occurs
Chromosome	A candidate solution encoding the global task assignment
MutatedChromosome	Chromosome after the mutation operation
v1, v2	Vectors holding indices of unresolved duplicates during crossover
L	Number of tasks or chromosome length
Sites	Set of industrial inspection sites
Robots	Set of available robots
Capabilities	Dictionary associating each robot with its measurement capabilities
m	Measurement type required for a task
r_k	Index or identifier of a specific robot
CompatibleRobots	Set of robots capable of performing measurement m
PopulationSize_Phase1	Number of chromosomes in global optimization phase
MaxGenerations_Phase1	Maximum iterations for global genetic optimization
PopulationSize_Phase2	Number of routes in local route optimization phase
MaxGenerations_Phase2	Maximum iterations in local route optimization phase
Population	Set of chromosomes in the current generation
Fitness	Quality score assigned to a chromosome or route
MatingPool	Subset of selected parents for reproduction
LocalPopulation	Population of route permutations for a robot in Phase 2
Route	A specific visiting order of assigned sites for a robot
OffspringRoutes	New set of routes after crossover and mutation in Phase 2
OptimalChromosome	Best chromosome representing the optimal task allocation and planning
BestRoute	Best visiting route assigned to a robot after local optimization

Sites	1	3	2	1	4	2	2	3
Measures	2	1	3	3	2	2	1	3
Robots	1	2	1	1	1	1	2	2

Fig. 4 Illustration of the proposed chromosome structure. Each column encodes site, measurement and robot respectively

resulting plan, robot r_1 visits site s_1 followed by site s_2 , and then site s_4 while robot r_2 proceeds from site s_3 to site s_2 .

After defining the chromosome structure, an initialization approach is implemented to generate the first population of the proposed GA. Typically, this population can be formed arbitrarily or through manual seeding when prior knowledge of the optimization problem is available. In this work, all chromosomes of the initial population are randomly initialized to ensure genetic diversity, which enhances the algorithm's ability to explore the solution space and prevents premature convergence to sub-optimal solutions. However,

to avoid unsuitable solutions, the three rows of chromosomes are generated as follows:

- *Row 1:* The number i corresponding to the site s_i is repeated as many times as there are measurements to be taken at that site.
- *Row 2:* For each number i on row 1, a number j on row 2 is associated if the measurement m_j was scheduled at the site s_i .
- *Row 3:* For each number j on row 2, a random number k on row 3 is associated if the robot r_k can perform the measurement m_j .

Phase-1 chromosomes are $3 \times L$ integer matrices whose rows encode site, measurement and robot. During initialization, the robot gene r in each column is drawn exclusively from $\mathcal{R}_m = \{r \mid m \in C_r\}$, so every column is capability-feasible by construction.

Evaluation and selection

A genetic algorithm generally requires an evaluation function to assess the relevance of the solutions. We denote this as the objective function. Given that our goal is to minimize the total distance traveled by robots during their inspection tasks, the objective function is expressed as follows:

$$f = \sum_{r=1}^R \sum_{i=0}^N \sum_{j=0}^N d_{ij} x_{ijr}. \quad (10)$$

To select individuals for the next generation, we use the binary tournament method, ensuring that the fittest candidates have a higher chance of being chosen. In this selection method, two individuals are randomly selected and their fitness values are compared. The individual with the lowest objective function value is selected for the next generation. This process is repeated until the new population is filled.

Genetic operators

Crossover and mutation are crucial for the proper functioning of the genetic algorithm. The crossover operator combines the characteristics of two parents to create new individuals, while mutation introduces variations by randomly altering genes. Although many crossover and mutation operators exist in the literature [11, 38, 39] (single-point crossover, two-point crossover, cycle crossover, order based crossover, bit-flip mutation, Gaussian mutation, and power mutation), these standard operators are not well suited to our application. Indeed, their limitations in handling the specific encoding used, coupled with their propensity to generate infeasible solutions, can severely impact the algorithm's performance and lead to poor convergence. Thus, we propose customized genetic operators designed to handle integer encoding while ensuring the feasibility of solutions.

Description of the crossover mechanism

The proposed crossover mechanism is illustrated in Fig. 5 and involves the following steps:

- *Step 1:* Select a crossover point along the parent chromosomes.
- *Step 2:* Preserve all columns before the crossover point.
- *Step 3:* Complete the offspring using columns from the other parent without duplicating existing columns corresponding to the same site and measurement.

The detailed procedure for the crossover operator is presented in Algorithm 1.

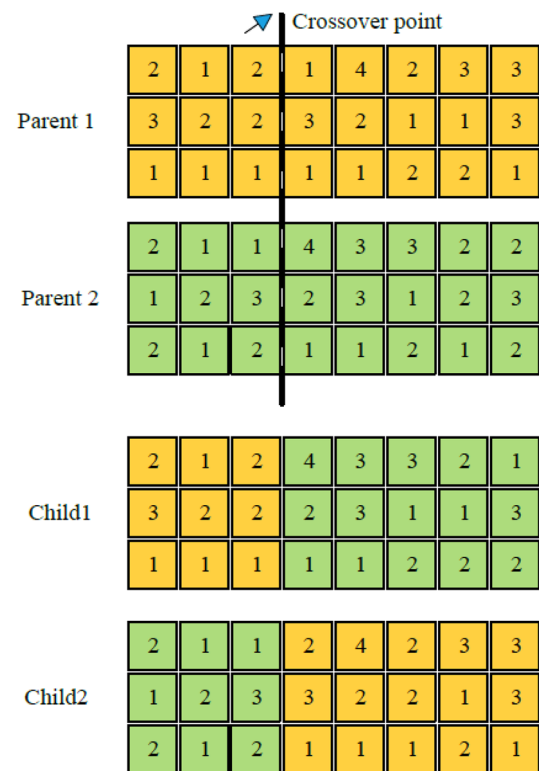


Fig. 5 Proposed crossover operator

Description of the mutation mechanism

We also introduce a new mutation operator, illustrated in Fig. 6. This operator follows the following steps:

- *Step 1:* Randomly select two genes (two columns: yellow and green) from the parent chromosome.
- *Step 2:* Swap the two selected genes.
- *Step 3:* Update the robots assigned to the tasks performed by the selected genes to other robots equipped with the necessary sensors.

The detailed mutation procedure is provided in Algorithm 2.

Phase 2: refinement of task planning

At the end of phase 1, tasks are defined for each robot and an initial scheduling is obtained. Thus, in this second step, the genetic algorithm is applied individually to each robot to optimize the sequence of tasks assigned to it, with the goal of minimizing the total distance traveled by all robots. To better illustrate this step, we consider an example with a larger number of sites (8 sites: $s_1, s_2, \dots, s_8$), each having a maximum of four possible measurements (m_1, m_2, m_3, m_4) and three robots (r_1, r_2, r_3) to perform the different mea-

Algorithm 1 Proposed crossover operator

Input: Parent1, Parent2, P_c : crossover probability
Output: Offspring1, Offspring2

```

1: if RandomNumber()  $\geq P_c$  then
2:   Offspring1=Parent1
3:   Offspring2=Parent2
4: else
5:   Select a random crossover point: CrossoverPoint
6:   Create empty offspring chromosomes: Offspring1, Offspring2
7:   Create two empty vectors: v1, v2
8:   L=LENGTH(Parent1)
9:   for  $i \leftarrow 1$  to CrossoverPoint do
10:    Offspring1.Columns[i]  $\leftarrow$  Parent1.Columns[i]
11:    Offspring2.Columns[i]  $\leftarrow$  Parent2.Columns[i]
12:   end for
13:   for  $i \leftarrow$  CrossoverPoint + 1 to L do
14:    Column1  $\leftarrow$  Parent2.Columns[i]
15:    Column2  $\leftarrow$  Parent1.Columns[i]
16:    if Column1 is not duplicate in Offspring1 then
17:      Offspring1.Columns[i]  $\leftarrow$  Column1
18:    else
19:      Append  $i$  to v1
20:    end if
21:    if Column2 is not duplicate in Offspring2 then
22:      Offspring2.Columns[i]  $\leftarrow$  Column2
23:    else
24:      Append  $i$  to v2
25:    end if
26:   end for
27:   k1=1;
28:   k2=1;
29:   for  $i \leftarrow 1$  to L do
30:    Column1  $\leftarrow$  Parent2.Columns[i]
31:    Column2  $\leftarrow$  Parent1.Columns[i]
32:    if Column1 is not duplicate in Offspring1 then
33:      Offspring1.Columns[v1[k1]]  $\leftarrow$  Column1
34:      k1=k1+1
35:    end if
36:    if Column2 is not duplicate in Offspring2 then
37:      Offspring2.Columns[v2[k2]]  $\leftarrow$  Column2
38:      k2=k2+1
39:    end if
40:   end for
41: end if
42: return Offspring1, Offspring2

```

Algorithm 2 Proposed mutation operator

Input: Chromosome, Robots, Capabilities, P_m : mutation probability
Output: MutatedChromosome

```

1: if RandomNumber()  $\geq P_m$  then
2:   MutatedChromosome=Chromosome
3: else
4:   Randomly select two distinct column indices:  $i$  and  $j$ 
5:   Swap columns  $i$  and  $j$  (i.e., swap Site, Measurement, and Robot values across all 3 rows)
6:   for each index in  $\{i, j\}$  do
7:      $m \leftarrow$  Chromosome[2][index]
8:     CompatibleRobots  $\leftarrow \{r_k \mid m \in \text{Capabilities}[r_k]\}$ 
9:     Chromosome[3][index]  $\leftarrow$  RANDOMSELECT(CompatibleRobots)
10:   end for
11:   MutatedChromosome=Chromosome
12: end if
13: return MutatedChromosome

```

surements. Suppose that the best chromosome obtained at the end of phase 1 is the one shown in Fig. 7. This “mother chromosome” will be used to generate the initial population for each robot, which will then undergo selection, crossover, and mutation operations to evolve over successive generations. As we can see in Fig. 7, robot r_1 should visit all sites to perform the different measurements assigned to it during the first step, while robot r_2 will also visit all sites except site 7, and robot r_3 will not visit any. Thus, during the second step, only the robots to which tasks are assigned will be involved in the optimization process. As we can also observe from Fig. 7, in this second step, the structure of the chromosome is simplified and reduced to a single row in contrast to the structure with three rows used in the first phase.

In addition, the size of the chromosome varies from one robot to another, depending on the number of sites to be visited. Moreover, the mechanism for generating the initial population and the genetic operators are relatively simple to implement. For each robot, the genetic algorithm starts by generating an initial population composed of several possible routes, obtained by randomly permuting the order of the assigned sites at the end of phase one. The algorithm then applies the genetic operators. The crossover used is the modified cycle crossover (CX2) proposed by [40], which efficiently combines the characteristics of the parent solutions while respecting the restrictions on the visit sites. In terms of mutation, it involves a random exchange of positions between two sites in the route, introducing variations that promote the exploration of new solutions and help escape local optima. Finally, as in the first phase, we use tournament selection.

We detail the full enhanced genetic algorithm (EGA) proposed to tackle the multi-robot task allocation problem. The provided pseudocode (Algorithm 3) thoroughly outlines the essential stages of the approach: initialization, evaluation, selection, crossover, mutation, and population update. Each

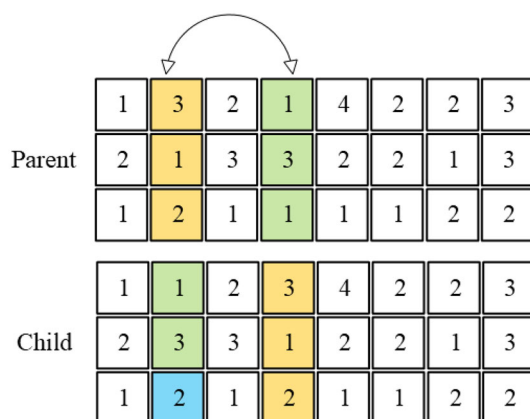


Fig. 6 Proposed mutation operator

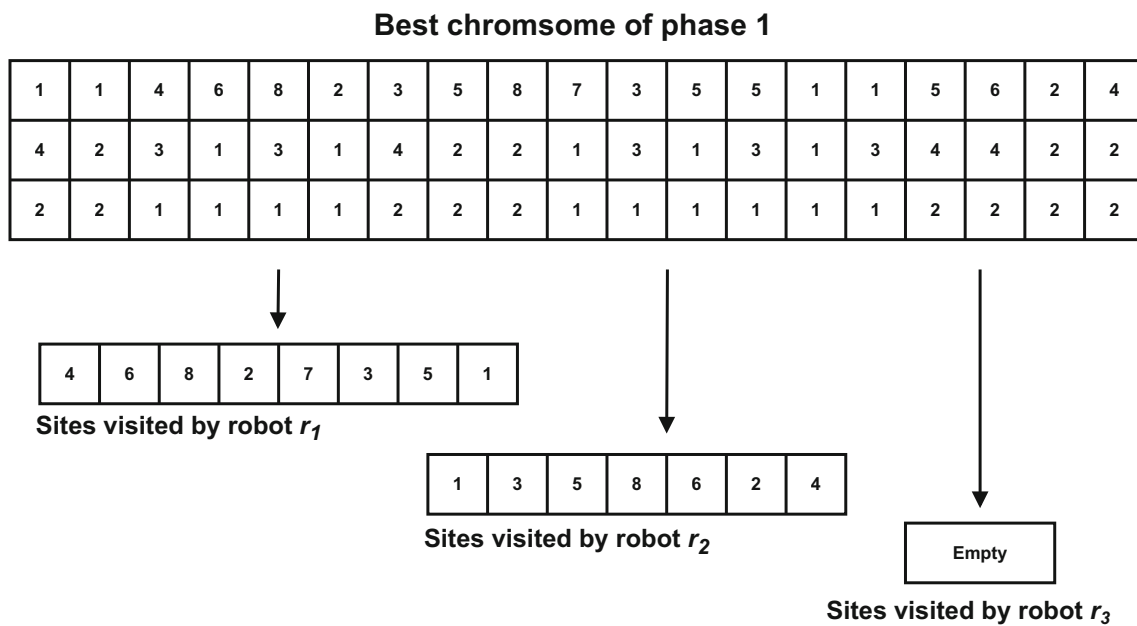


Fig. 7 Structure of chromosome in the Phase 2 of proposed EGA

stage utilizes previously described operators and processes to ensure an effective and feasible solution.

Experimental plan and use case

To rigorously evaluate the performance of the proposed enhanced genetic algorithm (EGA), we developed a realistic use case involving a multi-robot system designed for industrial site inspection. The experimental framework systematically explores various inspection scenarios, characterized by:

- **Inspection sites:** To assess the scalability of the proposed enhanced genetic algorithm (EGA) for multi-robot inspection, we conducted experiments on scenarios with varying numbers of inspection sites. Specifically, we considered instances with 5, 10, 15, 20, 25, 30, 35, 40, 45, and 50 sites. This comprehensive evaluation allows us to analyze the algorithm's performance and computational efficiency across small-, medium-, and large-scale inspection tasks, thereby providing insights into its robustness and practical applicability in industrial environments.
- **Robots:** Four heterogeneous robots (r_1, r_2, r_3, r_4), each equipped with distinct measurement capabilities (summarized in Table 2).
- **Measurements:** Four measurement types (m_1, m_2, m_3, m_4), with each site having predefined measurement needs. Each site is geographically dispersed with specified coordinates and requires multiple measurement types (see Table 3).

Table 2 Measurement capabilities of robots

Robot	Measurement capabilities
r_1	m_1, m_3
r_2	m_2, m_4
r_3	m_1, m_2
r_4	m_3

Performance metrics

To evaluate the performance of both our exact MILP and proposed EGA, we use the following key metrics:

- **Total travel distance :** The sum of the distances traveled by all deployed robots (see Eq. 10).
- **Computational time:** The CPU time (in seconds) that the solver or metaheuristic requires to find a feasible solution and (if applicable) prove optimality. For the MILP approach, this refers to the total run time until Gurobi [41] either terminates or reaches its preset time limit. For the EGA, the computational time refers to the total duration starting from the initialization of the population, followed by the iterative application of genetic operators, until the algorithm reaches the predefined maximum number of generations (MaxGenerations_Phase1 and MaxGenerations_Phase2).

Algorithm 3 Complete proposed EGA for multi-robot task allocation (with phase 1 and phase 2)

Input: PopulationSize_Phase1, MaxGenerations_Phase1, PopulationSize_Phase2, MaxGenerations_Phase2, Sites, Robots, Capabilities, P_c , P_m

Output: OptimalChromosome

PHASE 1: GLOBAL TASK ALLOCATION AND PLANNING
Initialization (See Section 4.1.1) :

```

1: Population ← GENERATEINITIALPOPULATION(PopulationSize_Phase1, Sites, Capabilities, Robots)
2: for Generation ← 1 to MaxGenerations_Phase1 do
  Evaluation:
3:   for all Chromosome in Population do
4:     Fitness ← EVALUATEFITNESS(Chromosome)
5:   end for
  Selection:
6:   MatingPool ← BINARYTOURNAMENTSELECTION(Population, Fitness)
  Crossover: (Algorithm 1)
7:   OffspringPopulation ← [ ]
8:   for  $i \leftarrow 1$  to PopulationSize_Phase1 by 2 do
9:     Parent1 ← MatingPool[i]
10:    Parent2 ← MatingPool[i+1]
11:    (Offspring1, Offspring2) ← PROPOSEDCROSSOVEROPERATOR(Parent1, Parent2,  $P_c$ )
12:    Append Offspring1 and Offspring2 to OffspringPopulation
13:  end for
  Mutation: (Algorithm 2)
14:  for all Offspring in OffspringPopulation do
15:    Offspring ← PROPOSEDMUTATIONOPERATOR(Offspring,  $P_m$ )
16:  end for
  Population Update:
17:  Population ← OffspringPopulation
18: end for
19: OptimalChromosome ← SELECTBESTINDIVIDUAL(Population)

```

PHASE 2: REFINEMENT OF TASK PLANNING

```

20: for all Robot  $r_k$  with tasks in OptimalChromosome do
21:   AssignedSites ← EXTRACTSITES(OptimalChromosome,  $r_k$ )
22:   LocalPopulation ← GENERATEPERMUTEDROUTES(AssignedSites, PopulationSize_Phase2)
23:   for LocalGeneration ← 1 to MaxGenerations_Phase2 do
24:     for all Route in LocalPopulation do
25:       Fitness ← EVALUATEROUTEFITNESS(Route)
26:     end for
27:     MatingPool ← TOURNAMENTSELECTION(LocalPopulation, Fitness)
28:     OffspringRoutes ← MODIFIEDCYCLECROSSOVER(MatingPool,  $P_c$ )
29:     OffspringRoutes ← MUTATEROUTES(OffspringRoutes,  $P_m$ )
30:     LocalPopulation ← OffspringRoutes
31:   end for
32:   BestRoute ← SELECTBESTROUTE(LocalPopulation)
33:   ASSIGNROUTETOROBOT(OptimalChromosome,  $r_k$ , BestRoute)
34: end for
35: return OptimalChromosome

```

Table 3 Details of inspection sites and measurement needs for a multi-robot system

Site ID	X coordinate	Y coordinate	Measurement needs
0	22	29	Depot
1	23	28	m_1, m_4
2	4	26	m_1, m_2
3	43	29	m_3, m_4
4	16	27	m_2, m_3
5	37	27	m_1, m_2, m_3, m_4
6	5	23	m_1, m_4
7	41	24	m_1
8	2	19	m_2, m_3
9	32	20	m_2, m_3, m_4
10	8	19	m_1, m_4
11	38	19	m_2
12	10	15	m_1, m_2, m_3, m_4
13	43	18	m_4
14	34	15	m_2, m_3
15	3	12	m_3
16	13	11	m_1, m_2
17	33	13	m_3, m_4
18	42	10	m_1, m_3, m_4
19	9	8	m_1, m_2, m_4
20	6	5	m_1, m_2, m_3, m_4
21	13	4	m_2, m_3, m_4
22	29	7	m_1, m_2
23	36	4	m_3, m_4
24	25	3	m_2, m_4
25	18	2	m_2, m_4
26	40	25	m_2, m_3
27	45	30	m_1, m_4
28	12	8	m_1, m_2, m_3
29	27	14	m_2, m_4
30	33	22	m_1, m_3
31	7	18	m_2
32	50	45	m_3, m_4
33	15	35	m_1, m_2
34	25	40	m_2, m_3, m_4
35	30	10	m_1, m_4
36	38	16	m_2, m_3
37	41	12	m_1, m_3, m_4
38	20	20	m_1, m_2, m_3
39	10	10	m_2, m_4
40	45	5	m_1

Table 3 continued

Site ID	X coordinate	Y coordinate	Measurement needs
41	5	45	m_3
42	28	33	m_1, m_4
43	34	27	m_2, m_3
44	42	35	m_1, m_2
45	16	40	m_2, m_3, m_4
46	12	12	m_1, m_4
47	37	37	m_1, m_2, m_3, m_4
48	29	19	m_2, m_3
49	32	32	m_1, m_3
50	40	40	m_2, m_4

The table lists each site (with site 0 as the depot) along with its coordinates and the measurements required at that site

- **Optimality gap (MIP gap %):** When using a MILP solver for exact methods, the solver maintains a lower bound (LB) and an upper bound (UB) on the objective value throughout the branch-and-bound process. Once the time limit is reached (or if an optimal solution is found earlier), the solver reports a MIP gap, defined as:

$$\text{MIP Gap\%} = 100 \times \frac{|\text{UB} - \text{LB}|}{|\text{UB}| + \varepsilon} \quad (11)$$

where UB is the best (lowest) objective value found so far (also known as the best incumbent), and LB is the global dual bound (or the best theoretical lower bound) obtained during branch-and-bound. The term ε , frequently set to 10^{-10} , is introduced to avoid division by zero when the best solution's objective value is extremely close to zero. A MIP gap of 0% indicates that the solver has proven global optimality of the incumbent, while a positive gap (e.g. 3%) implies the current solution may be up to 3% above the best theoretical bound. For an in-depth discussion of optimality gaps in MIP solvers, we refer to [42, 43]. As is common in operational research, a small MIP gap (e.g., below 1%) often indicates a near-optimal solution [44], which is acceptable in many industrial scenarios where absolute optimality is less critical than practical runtime.

Results and discussion

This section presents a comprehensive comparative assessment of the considered optimization approaches, namely the baseline GA (corresponding to the EGA restricted to Phase 1), the full EGA, PSO, MILP, and BL-SAEA. The comparison encompasses solution quality, convergence behavior,

computational efficiency, statistical significance, and scalability.

Performance enhancement through the two-phase EGA

To quantitatively evaluate the second phase's effectiveness in the proposed enhanced genetic algorithm (EGA), we performed an analysis comparing Phase 1 on its own to the combined two-phase approach (Phase 1 + Phase 2).

Figures 8 and 9 collectively assess the impact of the second phase in the EGA in terms of convergence behavior, solution quality, and robustness across increasing problem scales (from 10 to 50 sites).

Figure 8 presents convergence curves of the EGA over generations, comparing the standard GA (Phase 1 only, blue solid line) and the full EGA (Phase 1+2, red dashed line). For smaller-scale instances such as 10 sites (Fig. 8a), both configurations converge rapidly to nearly identical solutions. However, as the problem size increases (Fig. 8b–d), Phase 2 significantly improves the final solution quality, often after Phase 1 has plateaued. This behavior highlights the refinement capability of Phase 2, which exploits local improvements around promising solutions generated in Phase 1. These improvements become more pronounced as the problem grows in complexity, showcasing the scalability and effectiveness of the two-phase approach.

Figure 9 complements this analysis by presenting box-plots of the best values obtained across 30 independent runs for both configurations. Across all instance sizes, EGA Phase 1+2 consistently achieves better and more stable performance. The inter-quartile ranges for Phase 1 are notably wider, especially for 35 and 50 sites, indicating higher sensitivity to stochastic fluctuations. In contrast, Phase 1+2 not only reduces the variance of the best solutions but also significantly improves their central tendency (lower medians). This confirms that Phase 2 enhances the robustness of the algorithm by systematically refining and stabilizing the final solutions.

Together, these figures demonstrate that Phase 2 plays a critical role in enhancing both the solution quality and convergence reliability of the proposed EGA, particularly for medium-to-large scale industrial inspection scenarios. These findings support the adoption of a two-phase metaheuristic structure when robustness and precision are crucial.

Figure 10 illustrates the sensitivity of the EGA to population size across two phases for 50 inspection sites. In EGA Phase 1 (Fig. 10a), the performance is relatively stable, with a slight improvement at 200 individuals, suggesting that moderate sizes offer a good balance. In EGA Phase 1+2 (Fig. 10b), increasing the population to 3500 significantly improves results, while 5000 yields marginal gains with more variability. These findings highlight the importance of tuning

Fig. 8 Convergence behavior of the genetic algorithm (GA Phase 1) versus the EGA (Phase 1+2) for increasing problem sizes (10, 25, 35, and 50 sites)

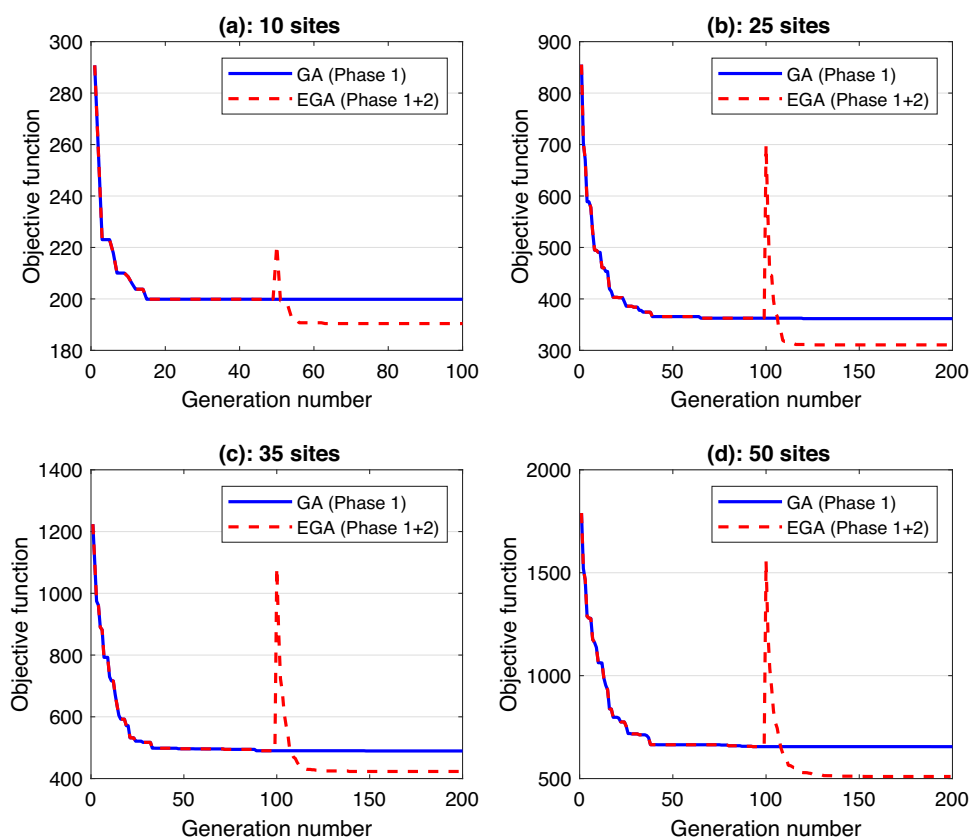


Fig. 9 Boxplot comparison of best objective values over 30 independent runs for GA (Phase 1) and EGA (Phase 1+2), across four problem sizes (10, 25, 35, 50)

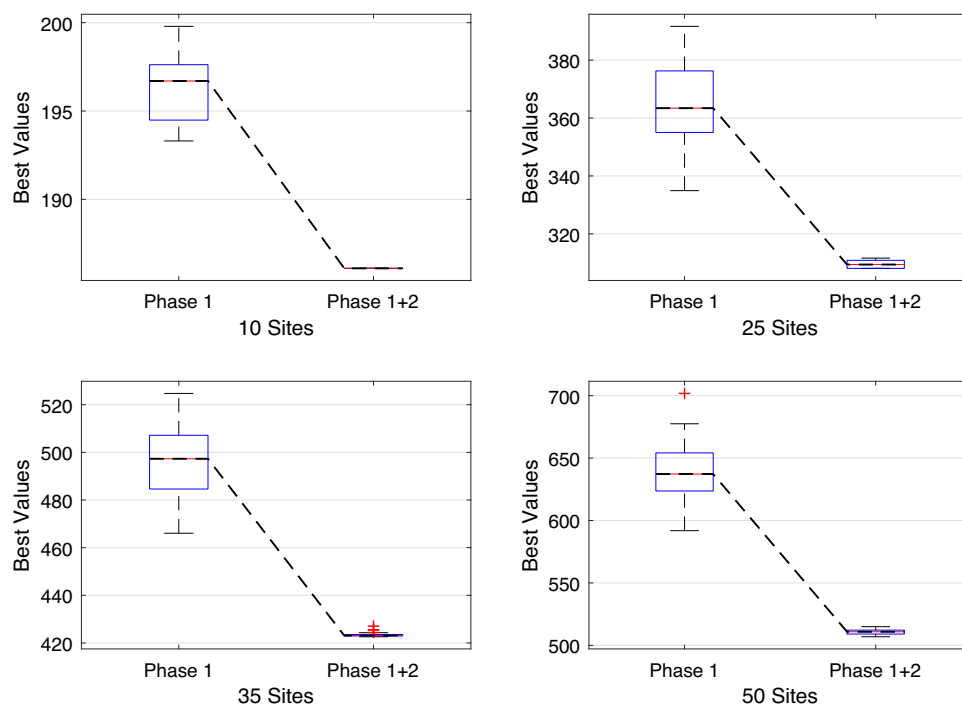


Fig. 10 Comparative sensitivity analysis of population size in phases 1 and 2 of the EGA for 50 inspection sites

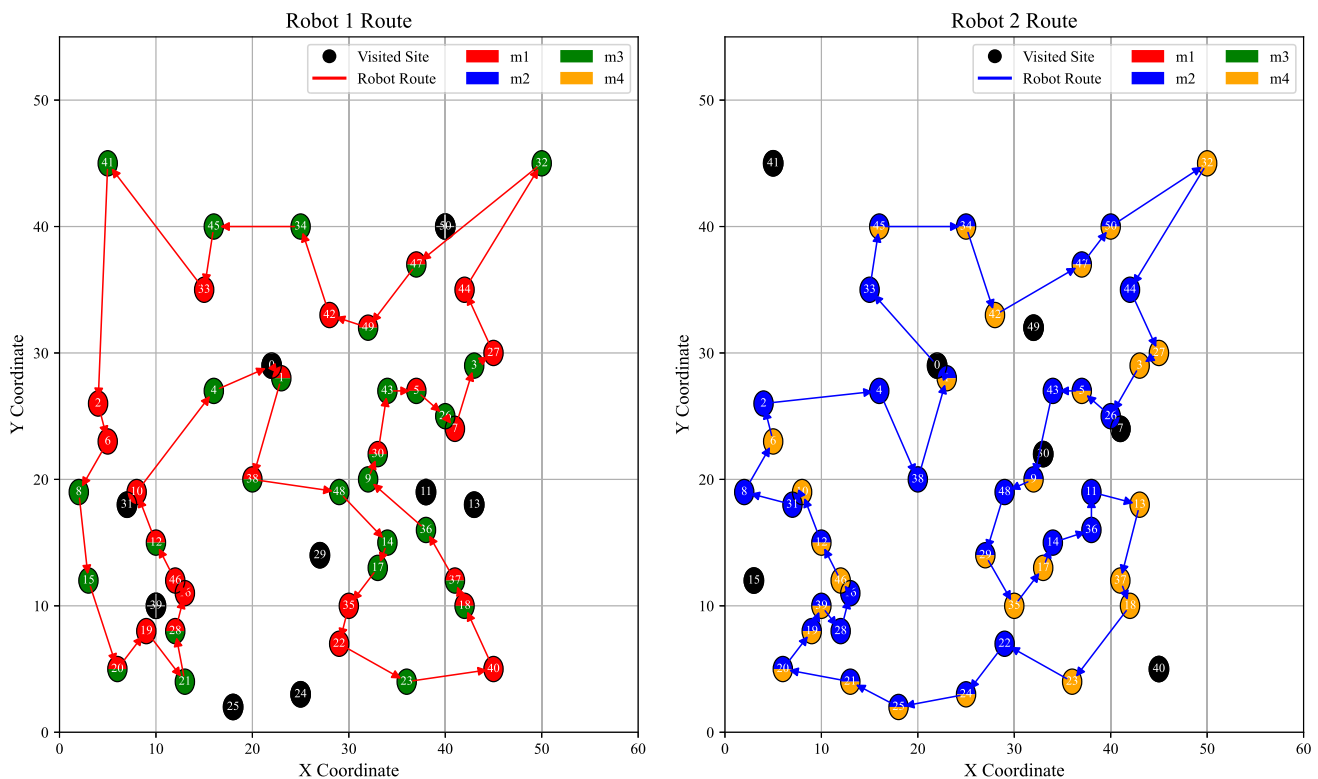
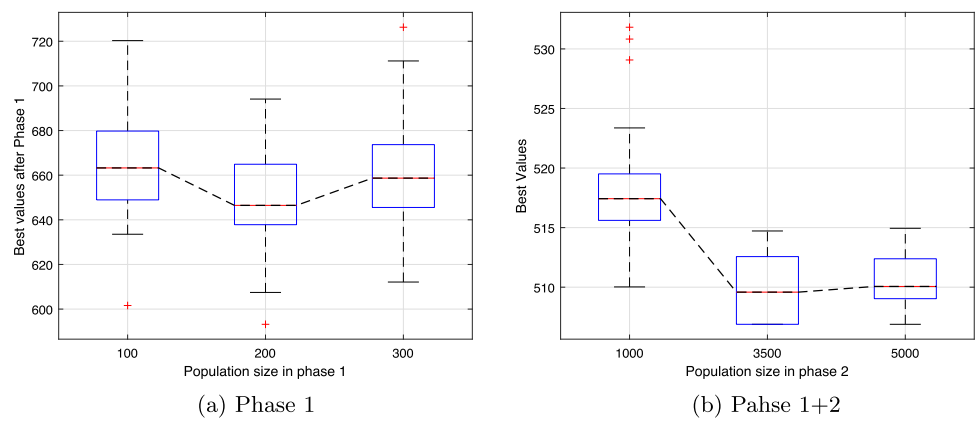


Fig. 11 Optimal robot trajectories generated by the proposed enhanced genetic algorithm (EGA) for a scenario with 50 inspection sites

population size to enhance solution quality without unnecessary computational cost.

Figure 11 illustrates an example of paths obtained by the proposed EGA for a scenario with 50 inspection sites. It provides a concrete example of how the measurements are assigned to each robot and their routes optimized. The total distance traveled by robot r_1 is 262.12, while robot r_2 covers a distance of 244.75, resulting in a combined total of 506.88.

Benchmark comparison of solution quality and execution time across methods

We evaluate the EGA's effectiveness by comparing its solution quality with three benchmark approaches: the baseline GA (phase 1 of EGA), PSO, and the exact MILP method under 60- and 600-s time limits. This comparison involves 30 benchmark instances of varying complexity, determined by the number of inspection sites and robots.

We further include a bi-level surrogate-assisted evolutionary algorithm (BL-SAEA), following the framework of [45]. Originally proposed for continuous domains, we adapt it to our combinatorial MRTA setting as follows:

- The upper level (UL) encodes task-to-robot assignments as integer vectors. The lower level (LL) evaluates each UL assignment by constructing the route for each robot, removing duplicate sites, and optimizing the visiting order via a 2-opt heuristic before summing all tour lengths. This realizes the two-level structure where UL searches assignments and LL provides the corresponding optimized routing cost.
- A Gaussian process regression (GPR) model serves as a surrogate for the UL fitness function. The kernel is a product of a constant term and a radial basis function plus a white noise term; hyperparameters are optimized via multiple restarts, and outputs are normalized. The surrogate is retrained whenever new evaluated individuals are added to the archive, which is capped at 200 entries.
- Candidate UL solutions are filtered before true LL evaluation using a non-dominated sorting procedure applied to the predicted mean objective values and their associated standard deviations. Only the non-dominated candidates, balancing predicted performance and uncertainty, are evaluated by the true bi-level procedure.
- An early stopping criterion is triggered when, after the archive limit is reached, the change in mean cost between consecutive archive segments falls below a threshold, indicating convergence.

Although the LL is combinatorial, the surrogate operates only at the UL on integer-encoded assignments. This design effectively prunes unpromising UL regions early, accelerating convergence in the combinatorial MRTA setting.

Table 5 presents the average total distances computed by each method. For metaheuristic approaches, results are reported as *mean ± standard deviation* over 30 independent runs, thereby capturing both the central tendency and the variability of each algorithm. For the MILP-based method, the best solutions obtained within each time limit are reported, along with the corresponding optimality gaps when available. This ensures a fair and consistent comparison between exact and heuristic methods.

To analyze the influence of robot heterogeneity, each problem instance-defined by a fixed number of inspection sites-was solved under three distinct robot configurations. The number of robots assigned to site coverage is reported in the first column of Table 5. For clarity, the values 2, 3, and 4 refer respectively to the following robot sets: (r_1, r_2) , (r_1, r_2, r_3) , and (r_1, r_2, r_3, r_4) . These configurations represent increasing levels of redundancy in sensing and operational capabilities, as summarized in Table 4. This experimental

Table 4 Robot configurations used to evaluate the influence of heterogeneity and capability overlap

# Robots	Robot set	Capability overlap level
2	r_1, r_2	No overlap
3	r_1, r_2, r_3	Moderate overlap
4	r_1, r_2, r_3, r_4	High overlap

design enables the evaluation of how the degree of capability overlap among heterogeneous robots affects task allocation performance and overall solution quality.

The results presented in Table 5 demonstrate the clear superiority of the proposed EGA in terms of solution quality and robustness. Across all tested problem sizes, EGA consistently outperforms the standard GA and the PSO, yielding lower average objective values with reduced variability. While PSO achieves convergence times comparable to GA, it fails to reach high-quality solutions, especially for larger instances exceeding 20 inspection sites. This limitation stems from PSO's original design for continuous optimization problems, resulting in premature convergence when applied to the discrete, combinatorial nature of multi-robot task allocation. The swap-based PSO variant used [46] partially mitigates this issue but remains outperformed by EGA.

The inclusion of BL-SAEA in the comparison provides additional insights into surrogate-assisted search performance. While its solution quality is generally slightly below that of EGA in most large-scale cases, BL-SAEA offers considerably shorter runtimes in some configurations, making it an attractive alternative when computational budget is constrained. In small- and medium-scale instances, its performance is often competitive with EGA, especially when problem structure allows the surrogate model to effectively guide the search.

Compared to the exact MILP solver, EGA delivers competitive or superior solutions for small to medium-sized problems, and significantly outperforms MILP on larger instances (40 to 50 sites), achieving up to 20–25% improvement in objective value. This advantage is attributed to MILP's computational limitations under strict time constraints (60 s and 600 s). BL-SAEA also occasionally surpasses the MILP results under the 60 s limit, particularly in mid-scale configurations, demonstrating the capability of surrogate-based methods to approximate high-quality solutions under tight time restrictions.

Furthermore, EGA maintains low standard deviations, indicating high consistency across multiple runs and configurations. This robust performance underlines EGA as a scalable and effective heuristic for real-world multi-robot task allocation, where near-optimal solutions and reasonable computational times are essential. BL-SAEA exhibits slightly higher variability in some large-scale cases, reflect-

Table 5 Comparison of solution quality and execution times for GA, EGA, PSO, BL-SAEA, and MILP under 60 s and 600 s limits

(Sites, robots)	GA		EGA		PSO		BL-SAEA		MILP60 s		MILP600 s	
	Solution	τ [s]	Solution	τ [s]	Solution	τ [s]	Solution	τ [s]	Solution	τ [s]	Solution	τ [s]
(5,2)	157.61 $\pm$ 0.17	0.05	157.44 $\pm$ 0.00	0.06	157.45 $\pm$ 0.03	0.14	157.44 $\pm$ 0.00	1.89	157.44 (0.00%)	0.03	157.44 (0.00%)	0.04
(5,3)	134.16 $\pm$ 0.04	0.19	134.14 $\pm$ 0.00	0.22	136.89 $\pm$ 3.73	0.23	137.47 $\pm$ 6.25	20.05	134.14 (0.00%)	0.04	134.14 (0.00%)	0.04
(5,4)	134.23 $\pm$ 0.09	0.63	134.14 $\pm$ 0.00	0.65	149.83 $\pm$ 11.67	0.25	145.17 $\pm$ 12.54	6.45	134.81 (0.00%)	0.04	134.81 (0.00%)	0.04
(10,2)	197.66 $\pm$ 3.62	1.76	186.09 $\pm$ 0.00	1.85	203.97 $\pm$ 8.66	2.47	186.09 $\pm$ 0.00	2.74	186.09 (0.00%)	0.24	186.09 (0.00%)	0.33
(10,3)	197.53 $\pm$ 3.90	1.91	186.30 $\pm$ 0.00	1.98	252.05 $\pm$ 11.06	2.51	187.26 $\pm$ 2.49	19.37	188.92 (0.00%)	2.35	188.92 (0.00%)	2.63
(10,4)	198.97 $\pm$ 3.65	2.12	186.09 $\pm$ 0.00	2.19	278.62 $\pm$ 11.14	2.55	229.08 $\pm$ 14.57	3.72	191.43 (0.00%)	4.27	191.43 (0.00%)	4.55
(15,2)	249.07 $\pm$ 9.39	2.84	221.56 $\pm$ 0.10	3.14	311.73 $\pm$ 24.80	5.25	221.54 $\pm$ 0.00	1.80	221.54 (0.00%)	1.31	221.54 (0.00%)	1.82
(15,3)	233.91 $\pm$ 6.24	5.93	221.56 $\pm$ 0.10	6.35	380.92 $\pm$ 17.74	4.59	232.74 $\pm$ 12.34	9.44	224.37 (11.08%)	60.06	224.37 (3.58%)	600.07
(15,4)	241.96 $\pm$ 6.17	6.71	221.62 $\pm$ 0.19	7.16	411.91 $\pm$ 14.03	5.24	299.83 $\pm$ 14.46	2.84	226.88 (11.33%)	60.05	226.88 (11.33%)	600.10
(20,2)	320.19 $\pm$ 10.34	7.09	283.00 $\pm$ 0.82	8.77	535.47 $\pm$ 33.27	10.29	283.14 $\pm$ 0.00	3.17	282.06 (0.00%)	18.32	282.06 (0.00%)	17.79
(20,3)	306.59 $\pm$ 9.21	10.81	282.88 $\pm$ 1.15	13.06	602.71 $\pm$ 20.83	10.06	312.46 $\pm$ 14.01	12.39	285.14 (13.86%)	60.05	285.13 (10.62%)	600.15
(20,4)	307.95 $\pm$ 8.07	10.92	283.15 $\pm$ 1.40	12.99	648.71 $\pm$ 21.83	10.31	403.47 $\pm$ 15.75	3.11	290.94 (15.66%)	60.09	287.96 (9.44%)	600.31
(25,2)	363.42 $\pm$ 13.96	7.61	309.66 $\pm$ 1.31	9.64	710.24 $\pm$ 21.98	13.83	308.56 $\pm$ 0.00	4.32	308.08 (0.00%)	6.04	308.08 (0.00%)	5.98
(25,3)	362.83 $\pm$ 10.44	12.64	310.66 $\pm$ 1.19	14.86	802.27 $\pm$ 19.36	13.20	351.52 $\pm$ 16.75	10.97	310.91 (0.00%)	52.30	310.91 (0.00%)	53.71
(25,4)	367.33 $\pm$ 17.71	14.83	310.46 $\pm$ 1.54	16.95	832.86 $\pm$ 25.39	13.19	444.32 $\pm$ 16.13	3.16	313.42 (4.15%)	60.10	313.42 (0.00%)	174.97
(30,2)	376.08 $\pm$ 11.79	10.21	328.36 $\pm$ 0.64	12.86	853.68 $\pm$ 19.09	21.21	335.09 $\pm$ 0.00	6.62	333.80 (2.65%)	60.18	333.80 (0.00%)	142.63
(30,3)	378.65 $\pm$ 9.87	17.76	328.36 $\pm$ 0.89	20.51	945.78 $\pm$ 27.98	20.41	386.24 $\pm$ 15.29	12.59	336.63 (4.95%)	60.29	336.63 (4.95%)	600.35
(30,4)	371.34 $\pm$ 10.07	19.07	328.24 $\pm$ 0.63	21.83	1001.72 $\pm$ 25.65	22.27	485.03 $\pm$ 11.23	3.72	342.10 (9.27%)	60.10	342.10 (9.27%)	691.94
(35,2)	505.13 $\pm$ 11.33	14.39	423.88 $\pm$ 1.17	21.05	1120.79 $\pm$ 18.02	25.75	476.60 $\pm$ 0.00	13.11	425.44 (1.24%)	60.37	425.44 (0.00%)	185.74
(35,3)	500.27 $\pm$ 11.53	29.60	425.96 $\pm$ 2.54	36.02	1215.75 $\pm$ 24.94	24.51	514.36 $\pm$ 9.62	16.88	474.23 (13.86%)	60.12	474.23 (13.92%)	600.34
(35,4)	497.38 $\pm$ 13.46	29.18	426.63 $\pm$ 2.84	34.91	1258.70 $\pm$ 25.36	21.86	597.91 $\pm$ 13.86	4.63	448.06 (8.16%)	60.10	428.71 (2.40%)	600.17
(40,2)	542.63 $\pm$ 11.20	17.85	455.76 $\pm$ 1.73	25.81	1260.41 $\pm$ 19.73	30.74	493.17 $\pm$ 0.00	18.73	454.80 (1.94%)	60.19	454.80 (0.51%)	600.83
(40,3)	520.08 $\pm$ 17.83	35.70	458.95 $\pm$ 2.83	43.26	1373.36 $\pm$ 28.33	28.82	544.93 $\pm$ 8.82	19.63	472.41 (7.72%)	60.24	455.24 (2.92%)	600.21
(40,4)	521.32 $\pm$ 16.37	37.44	458.51 $\pm$ 1.82	44.75	1437.84 $\pm$ 30.87	45.72	638.70 $\pm$ 16.53	5.43	466.39 (6.31%)	60.07	458.07 (3.35%)	600.14
(45,2)	596.73 $\pm$ 17.83	22.33	500.78 $\pm$ 2.92	34.25	1490.04 $\pm$ 23.07	36.79	544.57 $\pm$ 0.00	23.10	510.75 (5.57%)	60.19	497.21 (0.00%)	188.43
(45,3)	581.01 $\pm$ 16.31	43.90	503.32 $\pm$ 3.76	56.01	1597.02 $\pm$ 33.46	58.42	604.60 $\pm$ 13.65	24.61	606.75 (22.02%)	60.27	510.10 (4.77%)	600.23
(45,4)	582.56 $\pm$ 13.69	43.53	503.43 $\pm$ 3.52	53.80	1657.50 $\pm$ 28.35	93.23	706.48 $\pm$ 12.34	7.49	654.33 (26.34%)	60.29	506.04 (3.09%)	600.39
(50,2)	629.01 $\pm$ 19.20	28.23	510.38 $\pm$ 2.58	51.11	1642.47 $\pm$ 21.16	44.05	521.30 $\pm$ 0.00	41.08	515.92 (3.06%)	60.38	508.69 (1.17%)	601.22
(50,3)	609.75 $\pm$ 13.92	56.20	511.97 $\pm$ 3.46	77.52	1780.36 $\pm$ 24.47	69.68	600.31 $\pm$ 12.81	37.93	659.57 (25.97%)	60.27	511.52 (3.59%)	600.53
(50,4)	610.70 $\pm$ 15.40	60.27	512.35 $\pm$ 3.60	82.91	1850.95 $\pm$ 26.15	86.98	736.67 $\pm$ 13.59	7.97	773.59 (36.92%)	60.42	666.13 (26.73%)	600.21

Results are mean $\pm$ standard deviation over 30 runs; MILP values include MIP gap

ing the stochastic nature of surrogate retraining and candidate filtering; however, its stability remains acceptable for practical use.

Importantly, increasing the number of robots—thereby increasing capability overlap—does not degrade EGA’s performance. Instead, EGA effectively exploits redundancy, selecting minimal subsets of robots to cover all measurement requirements while minimizing travel cost. For instance, in the smallest problem (5 sites), increasing robot count from 2 to 3 or 4 reduces cost significantly (from 157.44 to 134.14), demonstrating EGA’s adaptive task distribution capabilities despite increased problem complexity. A similar trend is observed for BL-SAEA, though the magnitude of improvement with more robots is generally smaller than for EGA, suggesting that EGA’s enhancement phase leverages redundancy more effectively.

In addition to the comparative results presented in Table 5, detailed performance metrics of EGA and GA, including phase-wise analysis, are provided in 10. The addition of BL-SAEA to these comparative results enriches the discussion by illustrating the trade-off space between absolute solution quality and runtime efficiency, offering valuable guidance for practitioners with varying computational constraints.

Statistical validation of the proposed EGA

We conducted a detailed statistical analysis to evaluate the reliability and effectiveness of the proposed EGA. This evaluation encompasses 95% confidence interval (CI) estimation, hypothesis testing, and the calculation of relative performance improvements. The analysis focuses on pairwise comparisons of EGA against other metaheuristic baselines (GA, PSO), the BL-SAEA method, and the MILP approach, which serves as the optimality reference.

Each problem instance, defined by the pair $(|\mathcal{S}|, |\mathcal{R}|)$ representing the number of inspection sites and robots, was solved in 30 independent runs for each algorithm. For every method, we report the sample mean $\bar{m}$ together with its associated 95% CI, computed as:

$$CI = \bar{m} \pm t_{\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}, \quad (12)$$

where s is the sample standard deviation, $n = 30$, and $t_{\alpha/2, n-1}$ is the critical value of the Student’s t -distribution at $\alpha = 0.05$.

Table 6 summarizes the results. It includes the mean objective values and confidence intervals for GA, EGA, and PSO, as well as the optimal or best feasible values returned by the MILP solver (with its MIP Gap when convergence is not reached). It also reports the performance of the BL-SAEA method, allowing for a broader comparative perspective on surrogate-assisted strategies in discrete MRTA contexts. The

relative improvement of EGA over each baseline is computed as:

$$\text{Improvement}_{A \rightarrow B} = \frac{A - B}{A} \times 100, \quad (13)$$

where A and B denote the average values of the baseline and improved methods, respectively.

The results show that:

- EGA consistently outperforms GA, with improvements reaching up to 18.86% in large-scale instances;
- EGA significantly surpasses PSO, particularly in complex scenarios, with gains exceeding 70%;
- Compared to BL-SAEA, EGA generally delivers superior or equivalent performance, especially in high-complexity cases, while BL-SAEA achieves competitive results with markedly lower runtimes;
- EGA delivers competitive or superior solutions compared to MILP in most large instances where MILP does not converge within the 600-second limit.

In a few specific configurations—(20, 2), (25, 2), (40, 2), and (50, 2)—EGA underperforms slightly relative to MILP, with relative differences below 1%. However, these near-optimal results are obtained with runtimes at least eight times shorter, underscoring EGA’s practical efficiency. Similarly, BL-SAEA occasionally approaches or matches EGA’s performance in certain medium-scale cases. For example, in instance (10, 2) both methods reach the same mean objective value of 186.09, but BL-SAEA completes in 2.74 s compared to 1.85 s for EGA. In instance (15, 2), the difference between the two is negligible (221.54 vs. 221.56), yet BL-SAEA is faster (1.80 s vs. 3.14 s).

To rigorously assess the impact of the enhancement phase (Phase 2) in the EGA, a statistical comparison was conducted between the full EGA, the baseline GA, PSO, and BL-SAEA, following the same experimental protocol of 30 independent runs per instance. Welch’s two-tailed t -tests were applied to assess statistical significance, while Cohen’s d measured effect size. An effect size of $d > 0.8$ is conventionally interpreted as large, indicating substantial improvements.

For small-scale instances such as (5, 3) and (5, 4), there is no statistically significant difference between EGA and GA ($p > 0.05$), indicating convergence to similarly high-quality solutions. However, from (10, 2) onward, EGA consistently achieves statistically significant improvements over both GA and PSO ($p < 0.0001$), with large to very large effect sizes ($d > 5$ in most medium and large cases). Against BL-SAEA, EGA also shows statistically significant gains in most large-scale cases, although the performance gap is narrower; for instance, in (30, 4) EGA improves the mean solution from

Table 6 Compact statistical comparison of GA, EGA, PSO, BL-SAEA, and MILP (600 s) across all problem instances

(Sites, robots)	GA (m $\pm$ CI)	EGA (m $\pm$ CI)	PSO (m $\pm$ CI)	BL-SAEA (m $\pm$ CI)	MILP (m, gap%)	EGA vs GA (% $\downarrow$)	EGA vs MILP (% $\downarrow$)	EGA vs PSO (% $\downarrow$)	EGA vs BL-SAEA (% $\downarrow$)
(5,2)	157.61 $\pm$ 0.06	157.44 $\pm$ 0.00	157.45 $\pm$ 0.03	157.44 $\pm$ 0.00	157.44 (0.00%)	0.11	0.00	0.06	0.00
(5,3)	134.16 $\pm$ 0.01	134.14 $\pm$ 0.00	136.89 $\pm$ 3.73	137.47 $\pm$ 2.33	134.14 (0.00%)	0.01	0.00	1.80	2.42
(5,4)	134.23 $\pm$ 0.03	134.14 $\pm$ 0.00	149.83 $\pm$ 11.67	145.17 $\pm$ 4.68	134.81 (0.00%)	0.07	0.50	10.42	7.60
(10,2)	197.66 $\pm$ 1.35	186.09 $\pm$ 0.00	203.97 $\pm$ 8.66	186.09 $\pm$ 0.00	186.09 (0.00%)	5.84	0.00	8.70	0.00
(10,3)	197.53 $\pm$ 1.46	186.30 $\pm$ 0.00	252.05 $\pm$ 11.06	187.26 $\pm$ 0.93	188.92 (0.00%)	5.68	1.39	26.08	0.51
(10,4)	198.97 $\pm$ 1.36	186.09 $\pm$ 0.00	278.62 $\pm$ 11.14	229.08 $\pm$ 5.44	191.43 (0.00%)	6.49	2.74	33.20	18.77
(15,2)	249.07 $\pm$ 3.51	221.56 $\pm$ 0.04	311.73 $\pm$ 24.80	221.54 $\pm$ 0.00	221.54 (0.00%)	11.02	0.01	28.87	-0.01
(15,3)	233.91 $\pm$ 2.33	221.56 $\pm$ 0.04	380.92 $\pm$ 17.74	232.74 $\pm$ 4.61	224.37 (3.58%)	5.27	1.34	41.80	4.80
(15,4)	241.96 $\pm$ 2.30	221.62 $\pm$ 0.07	411.91 $\pm$ 14.03	299.83 $\pm$ 5.40	226.88 (11.33%)	8.39	2.26	46.19	26.07
(20,2)	320.19 $\pm$ 3.86	283.00 $\pm$ 0.31	535.47 $\pm$ 33.27	283.14 $\pm$ 0.00	282.06 (0.00%)	11.56	-0.33	-0.33	0.05
(20,3)	306.59 $\pm$ 3.44	282.88 $\pm$ 0.43	602.71 $\pm$ 20.83	312.46 $\pm$ 5.23	285.13 (10.62%)	7.72	0.91	53.06	9.46
(20,4)	307.95 $\pm$ 3.01	283.15 $\pm$ 0.52	648.71 $\pm$ 21.83	403.47 $\pm$ 5.88	287.96 (9.44%)	8.10	1.83	56.38	29.82
(25,2)	363.42 $\pm$ 5.21	309.66 $\pm$ 0.49	710.24 $\pm$ 21.98	308.56 $\pm$ 0.00	308.08 (0.00%)	14.78	-0.49	-0.49	-0.36
(25,3)	362.83 $\pm$ 3.90	310.66 $\pm$ 0.45	802.27 $\pm$ 19.36	351.52 $\pm$ 6.25	310.91 (0.00%)	14.38	0.07	61.28	11.62
(25,4)	367.33 $\pm$ 6.61	310.46 $\pm$ 0.58	832.86 $\pm$ 25.39	444.32 $\pm$ 6.02	313.42 (0.00%)	15.48	0.90	62.72	30.12
(30,2)	376.08 $\pm$ 4.40	328.36 $\pm$ 0.24	853.68 $\pm$ 19.09	335.09 $\pm$ 0.00	333.80 (0.00%)	12.72	1.55	61.52	2.01
(30,3)	378.65 $\pm$ 3.69	328.36 $\pm$ 0.33	945.78 $\pm$ 27.98	386.24 $\pm$ 5.71	336.63 (4.95%)	13.03	2.24	65.28	14.99
(30,4)	371.34 $\pm$ 3.76	328.24 $\pm$ 0.23	1001.72 $\pm$ 25.65	485.03 $\pm$ 4.19	342.10 (9.27%)	11.57	3.94	67.27	32.32
(35,2)	505.13 $\pm$ 4.23	423.88 $\pm$ 0.44	1120.79 $\pm$ 18.02	476.60 $\pm$ 0.00	425.44 (0.00%)	16.10	0.35	62.16	11.06
(35,3)	500.27 $\pm$ 4.30	425.96 $\pm$ 0.95	1215.75 $\pm$ 24.94	514.36 $\pm$ 3.59	474.23 (13.92%)	14.91	10.19	65.02	17.18
(35,4)	497.38 $\pm$ 5.03	426.63 $\pm$ 1.06	1258.70 $\pm$ 25.36	597.91 $\pm$ 5.18	428.71 (2.40%)	14.22	0.49	66.10	28.64
(40,2)	542.63 $\pm$ 4.18	455.76 $\pm$ 1.72	1260.41 $\pm$ 19.73	493.17 $\pm$ 0.00	454.80 (0.51%)	16.02	-0.21	63.84	7.58
(40,3)	520.08 $\pm$ 17.83	458.95 $\pm$ 2.83	1373.36 $\pm$ 28.33	544.93 $\pm$ 3.29	455.24 (2.92%)	11.81	-0.83	66.61	15.79
(40,4)	521.32 $\pm$ 16.37	458.51 $\pm$ 1.82	1437.84 $\pm$ 30.87	638.70 $\pm$ 6.17	458.07 (3.35%)	12.04	-0.12	68.09	28.22
(45,2)	596.73 $\pm$ 17.83	500.78 $\pm$ 2.92	1490.04 $\pm$ 23.07	544.57 $\pm$ 0.00	497.21 (0.00%)	16.08	-0.70	66.43	8.04
(45,3)	581.01 $\pm$ 16.31	503.32 $\pm$ 3.76	1597.02 $\pm$ 33.46	604.60 $\pm$ 5.10	510.10 (4.77%)	13.38	1.29	68.51	16.76
(45,4)	582.56 $\pm$ 13.69	503.43 $\pm$ 3.52	1657.50 $\pm$ 28.35	706.48 $\pm$ 4.61	506.04 (3.09%)	13.62	0.52	69.63	28.74
(50,2)	629.01 $\pm$ 19.20	510.38 $\pm$ 2.58	1642.47 $\pm$ 21.16	521.30 $\pm$ 0.00	508.69 (1.17%)	18.86	-0.29	68.95	2.10
(50,3)	609.75 $\pm$ 13.92	511.97 $\pm$ 3.46	1780.36 $\pm$ 24.47	600.31 $\pm$ 4.79	511.52 (3.59%)	16.02	-0.10	71.27	14.71
(50,4)	610.70 $\pm$ 15.40	512.35 $\pm$ 3.60	1850.95 $\pm$ 26.15	736.67 $\pm$ 5.08	666.13 (26.73%)	16.06	23.11	72.35	30.44

Values are mean $\pm$ 95% CI for metaheuristics and optimal solution with MIP Gap for MILP

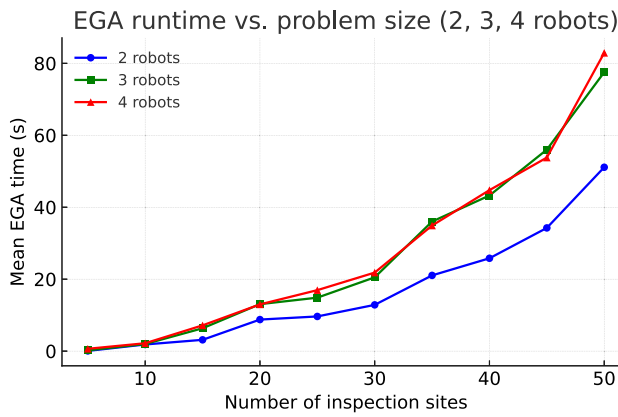


Fig. 12 Mean runtime of EGA versus $|\text{Sites}|$ for $|\text{Robots}| = 2$ (blue), 3 (green), and 4 (red)

485.03 ± 4.19 to 328.24 ± 0.23 (+32.32%), while in (40, 3) the improvement is 15.79% with $d = 15.79$.

As an illustrative example, for (50, 2), EGA reduces the mean objective from 629.01 ± 7.17 (GA) to 510.38 ± 0.96 , corresponding to an 18.86% improvement and a substantial effect size ($d = 8.66$). Compared to PSO, EGA achieves even larger gains (e.g., $d = 75.11$ for (50, 2)). In the same instance, BL-SAEA obtains a mean of 521.30 ± 0.00 , which is 2.10% higher than EGA but with a 41.08 s runtime versus 51.11 s for EGA, confirming its suitability when runtime is a critical factor.

Detailed statistical comparisons are reported in Table 8 in Appendix B. These findings confirm that Phase 2 of EGA substantially improves both the quality and robustness of solutions, particularly for complex or large-scale configurations. The inclusion of BL-SAEA in the comparative analysis highlights the complementary role of surrogate-assisted evolutionary search, especially when a balance between accuracy and computational cost is desired.

Scalability and computational-complexity analysis

The runtime scalability of the enhanced genetic algorithm (EGA) was evaluated on a benchmark grid with $|\text{Sites}| \in \{5, 10, \dots, 50\}$ and $|\text{Robots}| \in \{2, 3, 4\}$, using 30 independent runs per configuration. Figure 12 reports the *mean* runtime as a function of the number of sites for robot-team sizes of 2, 3, and 4.

Theoretical upper bound Let T_{EGA} denote the asymptotic runtime complexity of the Enhanced Genetic Algorithm, and let *Memory* denote its asymptotic space complexity in terms of population size and problem dimensions. In each of the *MaxGenerations_Phase1* generations, *PopulationSize_Phase1* chromosomes are evaluated.

A single fitness evaluation visits every site for each robot, which incurs a computational cost of:

$$\mathcal{O}(|\text{Sites}| \cdot |\text{Robots}|). \quad (14)$$

Therefore, the asymptotic time and memory complexities are:

$$T_{\text{EGA}} = \mathcal{O}(\text{MaxGenerations_Phase1} \times \text{PopulationSize_Phase1} \times |\text{Sites}| \times |\text{Robots}|), \quad (15)$$

$$\text{Memory} = \mathcal{O}(\text{PopulationSize_Phase1} \times |\text{Sites}|). \quad (16)$$

Empirical complexity law Assuming a power-law relationship between runtime and problem dimensions:

$$t_{\text{EGA}} = a |\text{Sites}|^b |\text{Robots}|^c, \quad (17)$$

we fitted the model using least squares over 45 empirical points ($\ln t$ vs. $\ln |\text{Sites}|$, $\ln |\text{Robots}|$). The fitted coefficients are:

$$a = 2.30 \times 10^{-3}, \quad b = 2.37 \pm 0.03, \quad c = 1.01 \pm 0.04, \quad R^2 = 0.99.$$

The R^2 value confirms that the model explains 99% of the runtime variance. Empirical growth is therefore quasi-quadratic in the number of sites and almost linear in the number of robots, which aligns with the theoretical bound.

Projection to larger instances Using the calibrated model, runtimes were extrapolated for instances up to $S = 200$ sites and $R \in \{2, 3, 4, 5, 10, 15, 20\}$:

$$\hat{t}(S, R) = 2.30 \times 10^{-3} \cdot S^{2.37} \cdot R^{1.01}. \quad (18)$$

For example: $\hat{t}(200, 2) \approx 1.3 \times 10^3$ s (≈ 22 min), and $\hat{t}(200, 10) \approx 6.7 \times 10^3$ s (≈ 112 min). Figure 13 illustrates these projections.

Scalability synopsis Theoretical and empirical results jointly indicate polynomial growth:

$$t_{\text{EGA}} \sim S^{2.37} \cdot R^{1.01}. \quad (19)$$

For the largest tested instance (50, 4), EGA completes in approximately 82 s. Extrapolations suggest that a 200-site mission would complete in ~ 22 min with a two-robot team, and that runtime scales almost linearly with the number of robots. These results confirm that EGA maintains practical scalability for large-scale industrial inspection scenarios, remaining computationally tractable even for considerably larger problem sizes.

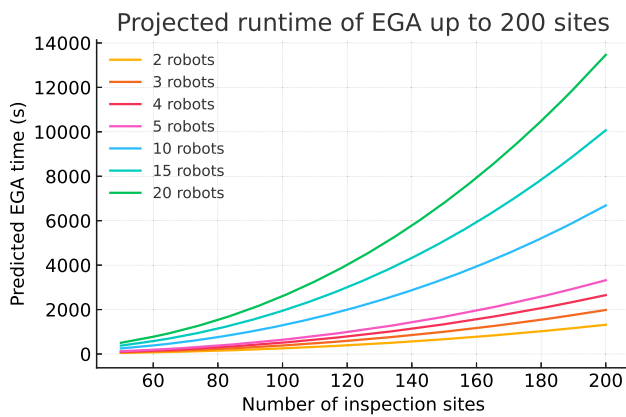


Fig. 13 Projected runtime of EGA up to $S = 200$ sites for various robot-team sizes, according to the fitted model $\hat{t} = 2.30 \times 10^{-3} S^{2.37} R^{1.01}$

EGA performance in a robot breakdown scenario

To further evaluate the adaptability of the proposed EGA algorithm, we conducted an experiment involving the sudden failure of a robot during task execution. Figure 14 illustrates

this scenario. In Fig. 14a, the nominal task allocation is shown for three robots (r_1 , r_2 , and r_3), each following clearly separated routes. At this stage, Robot r_1 has just completed site s_4 , Robot r_2 has serviced site s_1 , and Robot r_3 has also visited site s_4 before experiencing a failure.

Figure 14b depicts the re-optimized task distribution after the failure of r_3 , where its remaining tasks are efficiently reassigned to Robots r_1 and r_2 . The updated paths remain feasible and coherent with operational constraints. This scenario demonstrates the robustness of the EGA algorithm in handling unexpected disruptions, enabling continued task execution without requiring a complete re-optimization from scratch.

Conclusion and future work

This paper presented an enhanced genetic algorithm (EGA) for multi-robot task allocation and route planning in heterogeneous industrial inspection scenarios. By incorporating a two-phase methodology, first addressing global task allocation via customized integer-encoded crossover and mutation,

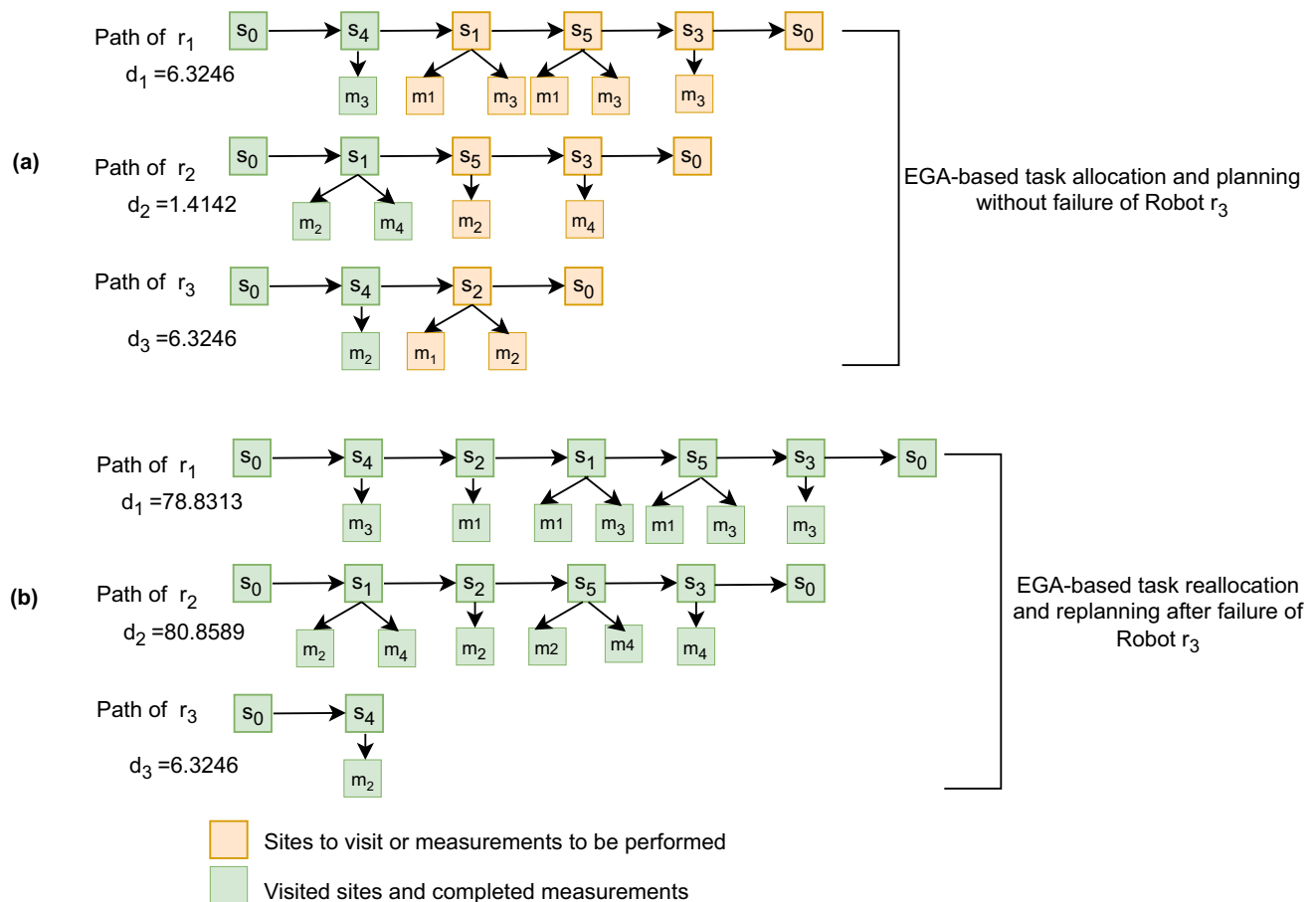


Fig. 14 Task allocation and scheduling before and after r_3 failure

then refining each robot's individual route. The proposed EGA effectively balances scalability with solution accuracy. Experimental comparisons against an exact mixed integer linear programming (MILP) formulation demonstrate that the EGA consistently yields near-optimal solutions (optimality gap below 1% for instances up to 30 sites, and below 3% up to 50 sites) while reducing computational time by over 90% in large-scale scenarios. These gains stem from the domain-specific chromosome encoding, specialized crossover/mutation operators, and the local refinement step, which collectively help avoid infeasible solutions and improve the final route quality. Furthermore, the inclusion of BL-SAEA in the comparative analysis confirms its ability to achieve competitive performance with markedly reduced runtimes, making it a viable alternative when computational efficiency is a primary constraint.

Beyond the raw performance metrics, the study also underscores how heterogeneity in robot capabilities—which may involve distinct sensor suites, energy constraints, or specialized measurement tools—complicates the allocation process. Standard metaheuristics can falter when naive operators generate infeasible or sub-par solutions. The proposed EGA addresses these limitations by embedding feasibility checks directly in its genetic operators, hence preserving solution quality throughout the evolutionary process. Finally, evidence from our results indicates that deploying more robots can reduce the total distance traveled, yet there are sometimes diminishing returns once basic coverage is ensured—highlighting the practical trade-offs between fleet size, operational cost, and task efficiency. The results with BL-SAEA also illustrate that surrogate-assisted search can complement more complex hybrid methods like EGA, especially for medium-scale scenarios where faster convergence is desirable without severely compromising accuracy.

Although this study focuses on industrial inspection, the modularity and structure of our proposed EGA approach allow for straightforward adaptation to other domains that share similar characteristics. For example, in warehouse logistics, robots must coordinate to pick and transport items with spatial constraints and dynamic task queues. Similarly, in search-and-rescue scenarios, tasks (e.g., area exploration, victim localization) must be assigned under uncertainty and evolving conditions.

Future research directions will focus on integrating advanced multi-objective optimization techniques to simultaneously minimize travel distance, energy consumption, and inspection time. Additionally, incorporating uncertainty modeling for task durations, robot availability, and environmental conditions will be explored to enhance robustness. Finally, integrating human-robot collaboration elements, such as adaptive task reassignment based on operator fatigue and cognitive load, represents another promising avenue to improve overall system performance and operator safety in realistic industrial contexts. In parallel, further investigations will explore hybridization strategies that combine the strengths of EGA and surrogate-assisted methods such as BL-SAEA, aiming to leverage both high accuracy and reduced computational cost.

Appendix A: Detailed results of the EGA

See Table 7.

Table 7 Performance of the enhanced genetic algorithm (EGA) for multi-robot inspection

(Sites, robots)	Optimal distance (mean $\pm$ std)		Execution time (s) (Phase 1/Phase 1+2)	Population Size, # MaxGenerations (Phase 1/Phase 1+2)	Optimal distance (Best–Worst)	
	EGA (Phase 1)	EGA (Phase 1+2)			Best	Worst
(5,2)	157.6131 $\pm$ 0.1703	157.4423 $\pm$ 0.0000	0.0531/0.0557	(40,15)/(40,15)	157.4423	157.4423
(5,3)	134.1569 $\pm$ 0.0386	134.1447 $\pm$ 0.0000	0.1869/0.2153	(40,30)/(30,20)	134.1447	134.1447
(5,4)	134.2281 $\pm$ 0.0878	134.1447 $\pm$ 0.0000	0.6296/0.6486	(100,50)/(30,20)	134.1447	134.1447
(10,2)	197.6585 $\pm$ 3.6191	186.0941 $\pm$ 0.0000	1.7641/1.8455	(150,100)/(150,50)	186.0941	186.0941
(10,3)	197.5273 $\pm$ 3.9027	186.2967 $\pm$ 0.0000	1.9143/1.9838	(150,100)/(150,50)	186.0941	186.0941
(10,4)	198.9729 $\pm$ 3.6499	186.0941 $\pm$ 0.0000	2.1163/2.1855	(150,100)/(150,50)	186.0941	186.0941
(15,2)	249.0658 $\pm$ 9.3940	221.5627 $\pm$ 0.0988	2.8388/3.1421	(150,100)/(400,100)	221.5446	222.0860
(15,3)	233.9055 $\pm$ 6.2438	221.5627 $\pm$ 0.0988	5.9266/6.3517	(300,100)/(400,100)	221.5446	222.0860
(15,4)	241.9617 $\pm$ 6.1675	221.6168 $\pm$ 0.1872	6.7063/7.1625	(300,100)/(400,100)	221.5446	222.0860
(20,2)	320.1896 $\pm$ 10.3393	282.9973 $\pm$ 0.8179	7.0888/8.7713	(200,100)/(1200,100)	282.0633	284.6659
(20,3)	306.5938 $\pm$ 9.2084	282.8770 $\pm$ 1.1480	10.8076/13.0559	(300,100)/(1200,100)	282.0633	284.3638
(20,4)	307.9459 $\pm$ 8.0672	283.1480 $\pm$ 1.4044	10.9220/12.9955	(300,100)/(1200,100)	282.3095	283.8299
(25,2)	363.4167 $\pm$ 13.9609	309.6627 $\pm$ 1.3095	7.6109/9.6407	(200,100)/(1200,100)	308.0768	311.6119
(25,3)	362.8257 $\pm$ 10.4355	310.6639 $\pm$ 1.1945	12.6439/14.8638	(300,100)/(1200,100)	308.6181	311.1249
(25,4)	367.3310 $\pm$ 17.7054	310.4591 $\pm$ 1.5417	14.8297/16.9512	(300,100)/(1200,100)	308.6181	312.1933
(30,2)	376.0751 $\pm$ 11.7885	328.3567 $\pm$ 0.6400	10.2103/12.8619	(200,100)/(1200,100)	327.8247	330.4969
(30,3)	378.6509 $\pm$ 9.8717	328.3612 $\pm$ 0.8854	17.7596/20.5098	(300,100)/(1200,100)	327.8247	330.4227
(30,4)	371.3441 $\pm$ 10.0689	328.2423 $\pm$ 0.6265	19.0728/21.8335	(300,100)/(1200,100)	327.8247	331.7385
(35,2)	505.1300 $\pm$ 11.3261	423.8779 $\pm$ 1.1739	14.3928/21.0470	(200,100)/(2000,100)	422.6150	427.5009
(35,3)	500.2675 $\pm$ 11.5250	425.9593 $\pm$ 2.5414	29.6048/36.0249	(400,100)/(2000,100)	422.6150	428.5316
(35,4)	497.3756 $\pm$ 13.4577	426.6327 $\pm$ 2.8359	29.1798/34.9128	(400,100)/(2000,100)	422.6150	430.9529
(40,2)	542.6323 $\pm$ 11.1982	455.7615 $\pm$ 1.7261	17.8507/25.8098	(200,100)/(2000,100)	453.0872	459.7766
(40,3)	520.0828 $\pm$ 17.8335	458.9485 $\pm$ 2.8307	35.7031/43.2585	(400,100)/(2000,100)	453.0872	464.3555
(40,4)	521.3230 $\pm$ 16.3704	458.5090 $\pm$ 1.8228	37.4364/44.7497	(400,100)/(2000,100)	457.1035	462.8722
(45,2)	596.7317 $\pm$ 17.8335	500.7774 $\pm$ 2.9163	22.3333/34.2465	(200,100)/(2500,100)	497.1291	505.9895
(45,3)	581.0091 $\pm$ 16.3063	503.3175 $\pm$ 3.7611	43.9010/56.0131	(400,100)/(2500,100)	497.1291	510.6910
(45,4)	582.5622 $\pm$ 13.6892	503.4328 $\pm$ 3.5247	43.5266/53.7996	(400,100)/(2500,100)	497.1291	509.8573
(50,2)	629.0062 $\pm$ 19.1961	510.3802 $\pm$ 2.5787	28.2294/51.1152	(200,120)/(3500,100)	506.8843	514.4365
(50,3)	609.7457 $\pm$ 13.9186	511.9725 $\pm$ 3.4620	56.2002/77.5218	(400,120)/(3500,100)	506.8843	518.3491
(50,4)	610.7033 $\pm$ 15.4042	512.3482 $\pm$ 3.6027	60.2711/82.9085	(400,120)/(3500,100)	506.8843	519.5858

Appendix B: Detailed statistical comparison between EGA, GA, PSO, and BL-SAEA

Author Contributions Ahmed Nait Chabane: conceptualization, methodology, software, validation, visualization, writing—original draft. Ouahib Guenounou: methodology, software, validation, visualization, writing—review and editing.

See Table 8.

Table 8 Statistical comparison of EGA against GA, PSO, and BL-SAEA using Welch's two-tailed *t*-tests and effect sizes (Cohen's *d*)

Instance	<i>p</i> -val (GA)	<i>d</i> (GA)	<i>p</i> -val (PSO)	<i>d</i> (PSO)	<i>p</i> -val (BL)	<i>d</i> (BL)
(5,2)	< 0.0001	1.414	0.0782	0.471	—	—
(5,3)	0.0104	0.707	< 0.0001	1.043	0.0067	0.750
(5,4)	< 0.0001	1.414	< 0.0001	1.901	< 0.0001	1.240
(10,2)	< 0.0001	4.520	< 0.0001	2.920	—	—
(10,3)	< 0.0001	4.072	< 0.0001	8.407	0.0434	0.550
(10,4)	< 0.0001	4.990	< 0.0001	11.747	< 0.0001	4.170
(15,2)	< 0.0001	4.143	< 0.0001	5.142	0.2823	0.280
(15,3)	< 0.0001	2.799	< 0.0001	12.704	< 0.0001	1.280
(15,4)	< 0.0001	4.660	< 0.0001	19.179	< 0.0001	7.650
(20,2)	< 0.0001	5.071	< 0.0001	10.729	0.3574	0.240
(20,3)	< 0.0001	3.613	< 0.0001	21.681	< 0.0001	2.980
(20,4)	< 0.0001	4.282	< 0.0001	23.634	< 0.0001	10.760
(25,2)	< 0.0001	5.422	< 0.0001	25.728	< 0.0001	1.190
(25,3)	< 0.0001	7.022	< 0.0001	35.844	< 0.0001	3.440
(25,4)	< 0.0001	4.524	< 0.0001	29.044	< 0.0001	11.680
(30,2)	< 0.0001	5.716	< 0.0001	38.895	< 0.0001	14.870
(30,3)	< 0.0001	7.177	< 0.0001	31.191	< 0.0001	5.340
(30,4)	< 0.0001	6.041	< 0.0001	37.121	< 0.0001	19.710
(35,2)	< 0.0001	10.088	< 0.0001	54.579	< 0.0001	63.720
(35,3)	< 0.0001	8.901	< 0.0001	44.554	< 0.0001	12.560
(35,4)	< 0.0001	7.273	< 0.0001	46.113	< 0.0001	17.120
(40,2)	< 0.0001	10.840	< 0.0001	57.456	< 0.0001	30.580
(40,3)	< 0.0001	4.789	< 0.0001	45.421	< 0.0001	13.130
(40,4)	< 0.0001	5.393	< 0.0001	44.787	< 0.0001	15.320
(45,2)	< 0.0001	7.510	< 0.0001	60.163	< 0.0001	21.210
(45,3)	< 0.0001	6.564	< 0.0001	45.937	< 0.0001	10.120
(45,4)	< 0.0001	7.917	< 0.0001	57.131	< 0.0001	22.380
(50,2)	< 0.0001	8.660	< 0.0001	75.106	< 0.0001	5.990
(50,3)	< 0.0001	9.641	< 0.0001	72.583	< 0.0001	9.420
(50,4)	< 0.0001	8.795	< 0.0001	71.716	< 0.0001	22.570

A *p* value less than 0.05 indicates significance. Cohen's *d* is computed as $d = \frac{|\bar{x}_1 - \bar{x}_2|}{s_p}$, with $s_p = \sqrt{(s_1^2 + s_2^2)/2}$. Thresholds: small (≥ 0.2), medium (≥ 0.5), large (≥ 0.8). For clarity, *p*-values $< 10^{-4}$ are reported as $p < 0.0001$. Instances with identical means and zero variance for both methods are marked “—” (indistinguishable).

Funding The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability All data generated or analyzed during this study are included in this published article.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Ethics approval This article does not contain any studies with human participants or animals performed by any of the authors.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Roveda L (2024) Editorial: human robot collaboration in industry 5.0: a human-centric ai-based approach. *Front Robot AI* 11:1511126. <https://doi.org/10.3389/frobt.2024.1511126>
- Soori M, Dastres R, Arezoo B, Jough FKG (2024) Intelligent robotic systems in industry 4.0: a review. *J Adv Manuf Sci Technol* 2024007-0 <https://doi.org/10.51393/j.jamst.2024007>
- Hémono P, Nait Chabane A, Sahnoun M (2025) Multi objective optimization of human robot collaboration: a case study in aerospace assembly line. *Comput Oper Res* 174:106874. <https://doi.org/10.1016/j.cor.2024.106874>
- Park J, Messing A, Ravichandar H, Hutchinson S (2023) Risk-tolerant task allocation and scheduling in heterogeneous multi-robot teams. In: 2023 IEEE/RSJ international conference on intelligent robots and systems (IROS), pp 5372–5379. <https://doi.org/10.1109/IROS55552.2023.10341837>
- Do H, Jang J, Kim J (2024) Heterogeneous multi-robot system mission planning with cooperative replenishment through data-driven rendezvous point selection. *Intell Serv Robot* 18:61–73. <https://doi.org/10.1007/s11370-024-00568-9>
- Nguyen AA, Jabbari F, Egerstedt M (2024) Resiliency through collaboration in heterogeneous multi-robot systems. *IEEE Open J Control Syst* 3:461–471. <https://doi.org/10.1109/OJCSYS.2024.3467991>
- Boccia M, Masone A, Sterle C, Murino T (2023) The parallel agv scheduling problem with battery constraints: a new formulation and a mathematical approach. *Eur J Oper Res* 307(2):590–603. <https://doi.org/10.1016/j.ejor.2022.10.023>
- Patil A, Bae J, Park M (2022) An algorithm for task allocation and planning for a heterogeneous multi-robot system to minimize the last task completion time. *Sensors* 22(15):5637. <https://doi.org/10.3390/s22155637>
- Nait Chabane A, Guenounou O (2024) Task allocation and planning for multi-robot system using an improved genetic algorithm. In: 2024 international conference of the African Federation of operational research societies (AFROS), pp 1–5. <https://doi.org/10.1109/AFROS62115.2024.11037105>
- Velasco L, Guerrero H, Hospitaler A (2024) A literature review and critical analysis of metaheuristics recently developed. *Arch Comput Methods Eng* 31(1):125–146
- Alhijawi B, Awajan A (2024) Genetic algorithms: theory, genetic operators, solutions, and applications. *Evol Intell* 17(3):1245–1256
- Li J, Cai Z, Li M, Huang W, Zhang Y (2023) Dynamic task allocation for heterogeneous multi-robot system under communication constraints. In: 2023 IEEE 6th information technology, networking, electronic and automation control conference (ITNEC), vol 6, pp 457–463. <https://doi.org/10.1109/ITNEC56291.2023.10082491>
- Aziz H, Pal A, Pourmiri A, Ramezani F, Sims B (2022) Task allocation using a team of robots. *Curr Robot Rep* 3(4):227–238
- Alirezazadeh S, Alexandre LA (2024) A survey on task allocation and scheduling in robotic network systems. *IEEE Internet Things J* 12(2):1484–1508. <https://doi.org/10.1109/JIOT.2024.3491944>
- Gerkey BP, Mataric MJ (2004) A formal analysis and taxonomy of task allocation in multi-robot systems. *Int J Robot Res* 23(9):939–954. <https://doi.org/10.1177/0278364904045564>
- Khan MT, Imanuel T, De Silva C (2010) Autonomous market-based multi-robot cooperation. In: 2010 international conference on intelligent and advanced systems. IEEE, pp 1–6
- Dias MB, Stentz A (2003) A comparative study between centralized, market-based, and behavioral multirobot coordination approaches. In: Proceedings 2003 IEEE/RSJ international conference on intelligent robots and systems (IROS 2003) (Cat. No. 03CH37453), vol 3. IEEE, pp 2279–2284
- Galati G, Primates S, Rizzo A (2023) Auction-based task allocation and motion planning for multi-robot systems with human supervision. *J Intell Robot Syst* 109(2):24
- Quinton F, Grand C, Lesire C (2023) Market approaches to the multi-robot task allocation problem: a survey. *J Intell Robot Syst* 107(2):29
- Poudel L, Elagandula S, Zhou W, Sha Z (2023) Decentralized and centralized planning for multi-robot additive manufacturing. *J Mech Des* 145(1):012003
- Chen Y, Rosolia U, Ames AD (2021) Decentralized task and path planning for multi-robot systems. *IEEE Robot Autom Lett* 6(3):4337–4344
- Chakraa H, Guérin F, Leclercq E, Lefebvre D (2023) Optimization techniques for multi-robot task allocation problems: review on the state-of-the-art. *Robot Auton Syst* 168:104492
- Notomista G, Mayya S, Emam Y, Kroninger C, Bohannon A, Hutchinson S, Egerstedt M (2021) A resilient and energy-aware task allocation framework for heterogeneous multirobot systems. *IEEE Trans Robot* 38(1):159–179
- Patra K, Sinha A, Guha A (2024) Online capability based task allocation of cooperative manipulators. *J Intell Robot Syst* 110(1):23
- Drenjanac D, Tomic SDK, Kühn E (2015) A semantic framework for modeling adaptive autonomy in task allocation in robotic fleets. In: 2015 IEEE 24th international conference on enabling technologies: infrastructure for collaborative enterprises. IEEE, pp 15–20
- Jiang Y, Pang X, Zhang Z, Jing H, Wei L, Su J, Zhang P (2024) Integrating intelligent sensors for safe uav distribution: design and evaluation of ranging system. *HighTech Innov J* 5(3):603–613
- Puentes K, Morales L, Pozo-Espin DF, Moya V (2024) Enhancing control systems with neural network-based intelligent controllers. *Emerg Sci J* 8(4):1243–1261

28. Li S, Shen L (2024) Seismic optimization design and application of civil engineering structures integrated with building robot system technology. *HighTech Innov J* 5(4):1118–1134
29. Shelkamy M, Elias CM, Mahfouz DM, Shehata OM (2020) Comparative analysis of various optimization techniques for solving multi-robot task allocation problem. In: 2020 2nd novel intelligent and leading emerging sciences conference (NILES), pp 538–543. <https://doi.org/10.1109/NILES50944.2020.9257967>
30. Liu X-F, Lin B-C, Zhan Z-H, Jeon S-W, Zhang J (2021) An efficient ant colony system for multi-robot task allocation with large-scale cooperative tasks and precedence constraints. In: 2021 IEEE symposium series on computational intelligence (SSCI). IEEE, pp 1–8
31. Kong X, Gao Y, Wang T, Liu J, Xu W (2019) Multi-robot task allocation strategy based on particle swarm optimization and greedy algorithm. In: 2019 IEEE 8th joint international information technology and artificial intelligence conference (ITAIC). IEEE, pp 1643–1646
32. Liao YY, Ryu K (2022) Genetic algorithm-based task allocation in multiple modes of human-robot collaboration systems with two cobots. *Int J Adv Manuf Technol* 119(11):7291–7309
33. Tolmidis AT, Petrou L (2013) Multi-objective optimization for dynamic task allocation in a multi-robot system. *Eng Appl Artif Intell* 26(5–6):1458–1468
34. Zhang L, Zhao J, Lamon E, Wang Y, Hong X (2025) Energy efficient multi-robot task allocation constrained by time window and precedence. *IEEE Trans Autom Sci Eng* 22:18162–18173. <https://doi.org/10.1109/TASE.2023.3312214>
35. Mammadzada R (2023) Various scheduling techniques used in human-robot collaborative system strategies for allocation and optimization of tasks. *J Mod Technol Eng* 8(3)
36. Shem-Tov E, Elyasaf A (2024) Deep neural crossover: a multi-parent operator that leverages gene correlations. In: Proceedings of the genetic and evolutionary computation conference, pp 1045–1053
37. Nunes E, Gini M (2017) A taxonomy for task allocation problems in multi-robot systems. *Auton Robot* 41(5):1189–1220
38. Guenounou O, Dahhou B, Chabour F (2015) Tsk fuzzy model with minimal parameters. *Appl Soft Comput* 30:748–757
39. Bolotbekova A, Hakli H, Beskirli A (2025) Trip route optimization based on bus transit using genetic algorithm with different crossover techniques: a case study in konya/türkiye. *Sci Rep* 15(1):2491
40. Hussain A, Muhammad YS, Nauman Sajid M, Hussain I, Mohamud Shoukry A, Gani S (2017) Genetic algorithm for traveling salesman problem with modified cycle crossover operator. *Comput Intell Neurosci* 2017(1):7430125
41. Gurobi Optimization, LLC (2024) Gurobi Optimizer Reference Manual. <https://www.gurobi.com>
42. Nemhauser GL, Wolsey LA (1988) Integer and combinatorial optimization. Wiley-Interscience, New York
43. Bixby RE (2002) Solving real-world linear programs: a decade and more of progress. *Oper Res* 50(1):3–15
44. Bixby ER, Fenelon M, Gu Z, Rothberg E, Wunderling R (1999) Mip: theory and practice closing the gap. In: IFIP conference on system modeling and optimization. Springer, pp 19–49
45. Jiang H, Chou K, Tian Y, Zhang X, Jin Y (2024) Efficient surrogate modeling method for evolutionary algorithm to solve bilevel optimization problems. *IEEE Trans Cybern* 54(7):4335–4347. <https://doi.org/10.1109/TCYB.2023.3309598>
46. Emambocus BAS, Jasser MB, Hamzah M, Mustapha A, Amphawan A (2021) An enhanced swap sequence-based particle swarm optimization algorithm to solve tsp. *IEEE Access* 9:164820–164836

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.